



**STOCK PRICE PREDICTION AND FORECASTING USING
MACHINE LEARNING
MINI PROJECT REPORT**

Submitted by

**SATHISH KUMAR M (20EUAI034)
THARUN S (20EUAI048)
VARUN S (20EUAI051)
VIKASS ARAVIND C V (20EUAI054)**

in partial fulfillment of the requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

**SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY
COIMBATORE
(An Autonomous Institution)**

APRIL 2023



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution. Affiliated to Anna University, Chennai)
Kuniamuthur, Coimbatore - 641 008



BONAFIDE CERTIFICATE

Certified that this mini project report titled **“STOCK PRICE PREDICTION AND FORECASTING USING MACHINE LEARNING”** is the bonafide work of **“SATHISH KUMAR M (20EUI034), THARUN S (20EUI048), VARUN S (20EUI051), VIKASS ARAVIND C V (20EUI054)”** who carried out the mini project work under my supervision.

SIGNATURE

Mr. G. S. PUGALENDHI M.E.,
SUPERVISOR,
ASSISTANT PROFESSOR

SIGNATURE

Dr. S.VENKATA LAKSHMI M.TECH., Ph.D.,
HEAD OF THE DEPARTMENT

Department of Artificial Intelligence and Data Science
Sri Krishna College of Engineering and Technology
Kuniamuthur,
Coimbatore.

This Mini Project report is submitted for Autonomous Mini Project Viva-Voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our sincere thanks to the management and **Dr. J. JANET M.E., Ph.D.**, Principal, Sri Krishna College of Engineering and Technology, Coimbatore for providing us the facilities to carry out this mini project work.

We are thankful to **Dr. S. VENKATA LAKSHMI M.TECH., Ph.D.**, Professor and Head, Department of Artificial Intelligence and Data Science, for her continuous evaluation and comments given during the course of the mini project work.

We express our deep sense of gratitude to our supervisor **Mr. G. S. PUGALENDHI M.E.**, Assistant Professor, Department of Artificial Intelligence and Data Science for his valuable advice, guidance and support during the course of our mini project work.

We express our heartfelt sense of gratitude and thanks to our beloved parents, family and friends who have helped during the mini project course.

ABSTRACT

Stock markets can often be one of the highly complex and most volatile places to invest. Statistical analysis of past stock performance and external factors play a major role in the decision to buy or sell stocks. These factors are all used to maximize profits. Stock price index forecasting has been a subject of great research for many years, and several algorithms have been proposed to simplify this complex task, but little success has been found so far. People have been using learning tools from diverse fields such as financial mathematics and machine learning to make trustworthy forecasting on such markets. However, the accuracy of such techniques had not been adequate until artificial neural network frameworks such as long short-term memory (LSTM) were utilized. In order to forecast stocks accurately, it is crucial to understand the context-specific dependence of stock prices on their past values. We treat the current forecast as a part of the training set for the next forecast and train the same LSTM. While classic ways of training cause more error when the forecast is made further away through the test period, our approach offers superior accuracy as the training increases when it proceeds through the testing period. The use of historical data which is more than ten years adds as an advantage. The forecasting accuracy of our approach is validated using time series forecasting of stock prices. This approach involves creation of three models which are Long Short Term Memory Networks (LSTM), Bidirectional - Long Short Term Memory Networks (Bi-LSTM) and Recurrent Neural Networks (RNN) and a Time-Series Forecasting Algorithm (FbProphet). The proposed approach gives us a minimum loss of 0.088%. This approach can be used in any type of prediction and forecasting of stocks.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
	TABLE OF CONTENTS	v
	LIST OF TABLES	ix
	LIST OF FIGURES	x
	LIST OF SYMBOLS AND ABBREVIATIONS	xi
1	INTRODUCTION	1
	1.1 STOCK PRICE PREDICTION	1
	1.2 RECURRENT NEURAL NETWORKS (RNN)	2
	1.3 LONG SHORT TERM MEMORY (LSTM)	3
	1.4 BIDIRECTIONAL - LONG SHORT TERM MEMORY (BI-LSTM)	4
	1.5 FBPROPHET	5
2	LITERATURE REVIEW	6
3	SYSTEM ANALYSIS	8
	3.1 PROBLEM STATEMENT	8
	3.2 EXISTING SYSTEM	8

3.2.1	LIMITATIONS OF EXISTING SYSTEM	9
3.3	PROPOSED SYSTEM	9
3.3.1	ADVANTAGE OF THE PROPOSED SYSTEM	10
3.4	REQUIREMENT ANALYSIS AND SPECIFICATION	10
3.4.1	REQUIREMENT DESCRIPTION	10
3.4.2	HARDWARE SPECIFICATIONS	17
3.4.3	SOFTWARE SPECIFICATIONS	17
4	SYSTEM DESIGN	18
4.1	SYSTEM DESIGN	18
4.1.1	INPUT DESIGN	18
4.1.2	OUTPUT DESIGN	19
4.2	DESIGN CONSTRAINTS AND STANDARDS	21
4.3	DESIGN ALTERNATIVES	21
4.4	MODULE DESCRIPTION	22
4.4.1	DETAILED DESIGN	22
4.5	DATA COLLECTION	22
4.6	DATA PRE-PROCESSING	23
4.7	TRAINING THE MODEL	23
4.8	WEB APPLICATION	24

5	IMPLEMENTATION	27
	5.1 FRONT END	27
	5.1.1 STREAMLIT	27
	5.2 BACK END	28
	5.2.1 PYTHON	28
	5.3 RESULTS	29
6	TESTING	32
	6.1 TESTING	32
	6.2 STRATEGIC APPROACH TO SOFTWARE TESTING	32
	6.3 UNIT TESTING	33
	6.4 WHITE BOX TESTING	33
	6.5 CONDITIONAL TESTING	33
	6.6 DATA FLOW TESTING	34
	6.7 LOOP TESTING	34
	6.8 TEST CASE	34
	6.8.1 TEST CASE 1	35
	6.8.2 TEST CASE 2	35
7	CONCLUSION AND FUTURE WORK	36
	REFERENCES	37

APPENDICES	39
APPENDIX - I	39
APPENDIX - II	47
BASE PAPER	49

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1	MODEL METRICS	31

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1	USE CASE DIAGRAM	19
2	ACTIVITY DIAGRAM	20
3	PROPOSED WORKFLOW	25
4	OVERALL WORKFLOW DIAGRAM	26
5	LOSS GRAPH OF RNN MODEL	29
6	LOSS GRAPH OF LSTM MODEL	29
7	LOSS GRAPH OF BI - LSTM MODEL	30
8	COMPARISON OF LOSS OF DIFFERENT MODELS	30
9	TEST CASE 1	35
10	TEST CASE 2	35
11	HOME PAGE	47
12	PREDICTION PAGE	47
13	ANALYSE PAGE	48
14	FBPROPHET PAGE	48

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
ML	MACHINE LEARNING
DL	DEEP LEARNING
LSTM	LONG SHORT TERM MEMORY
BI – LSTM	BI-DIRECTIONAL LONG SHORT TERM MEMORY
RNN	RECURRENT NEURAL NETWORKS
RMSE	ROOT MEAN SQUARED ERROR
MAPE	MEAN ABSOLUTE PERCENTAGE ERROR MEASURE
MLS LSTM	MULTI-LAYER SEQUENTIAL LONG SHORT TERM MEMORY
MLCA-LSTM	MULTI-SCALE LOCAL CUES AND HIERARCHICAL ATTENTION-BASED LSTM

CHAPTER – 1

INTRODUCTION

1.1 STOCK PRICE PREDICTION

Stock Market prediction and analysis is the act of trying to determine the future value of a company stock or other financial instrument traded on an exchange. Stock market is the important part of economy of the country and plays a vital role in the growth of the industry and commerce of the country that eventually affects the economy of the country. Both investors and industry are involved in stock market and wants to know whether some stock will rise or fall over certain period of time. The stock market is the primary source for any company to raise funds for business expansions. It is based on the concept of demand and supply. If the demand for a company's stock is higher, then the company share price increases and if the demand for company's stock is low then the company share price decrease. Due to involvement of many number of industries and companies, it contain very large sets of data from which it is difficult to extract information and analyse their trend of work manually. Stock market analysis and prediction will reveal the market patterns and predict the time to purchase stock. The successful prediction of a stock's future price could yield significant profit.

Machine learning is a method of data analysis that automates analytical model building. Using algorithms that iteratively learn from data, machine learning allows computers to find hidden insights without being explicitly programmed where to look. Application of Machine learning algorithms is generally focused on technical analysis but incorporation of the concepts of fundamental analysis into machine learning can be beneficial. This project tells about how various efforts have been taken in the application of Machine Learning to Stock forecasting and also suggests new potent ideas that can be worked upon. Recently, a lot of interesting work has been done in the area of applying Machine Learning Algorithms for analysing price patterns and predicting stock prices and index changes.

1.2 RECURRENT NEURAL NETWORKS (RNN)

A recurrent neural network (RNN) is a type of artificial neural network which uses sequential data or time series data. These deep learning algorithms are commonly used for ordinal or temporal problems, such as language translation, natural language processing (nlp), speech recognition, and image captioning; they are incorporated into popular applications such as Siri, voice search, and Google Translate. Like feedforward and convolutional neural networks (CNNs), recurrent neural networks utilize training data to learn. They are distinguished by their “memory” as they take information from prior inputs to influence the current input and output. While traditional deep neural networks assume that inputs and outputs are independent of each other, the output of recurrent neural networks depend on the prior elements within the sequence. While future events would also be helpful in determining the output of a given sequence, unidirectional recurrent neural networks cannot account for these events in their predictions.

Another distinguishing characteristic of recurrent networks is that they share parameters across each layer of the network. While feedforward networks have different weights across each node, recurrent neural networks share the same weight parameter within each layer of the network. That said, these weights are still adjusted in the through the processes of backpropagation and gradient descent to facilitate reinforcement learning.

Recurrent neural networks leverage backpropagation through time (BPTT) algorithm to determine the gradients, which is slightly different from traditional backpropagation as it is specific to sequence data. The principles of BPTT are the same as traditional backpropagation, where the model trains itself by calculating errors from its output layer to its input layer. These calculations allow us to adjust and fit the parameters of the model appropriately.

1.3 LONG SHORT TERM MEMORY (LSTM)

To solve the problem of Vanishing and Exploding Gradients in a Deep Recurrent Neural Network, many variations were developed. One of the most famous of them is the Long Short Term Memory Network(LSTM). In concept, an LSTM recurrent unit tries to “remember” all the past knowledge that the network is seen so far and to “forget” irrelevant data. This is done by introducing different activation function layers called “gates” for different purposes. Each LSTM recurrent unit also maintains a vector called the Internal Cell State which conceptually describes the information that was chosen to be retained by the previous LSTM recurrent unit.

LSTM networks are the most commonly used variation of Recurrent Neural Networks (RNNs). The critical component of the LSTM is the memory cell and the gates (including the forget gate but also the input gate), inner contents of the memory cell are modulated by the input gates and forget gates. Assuming that both of the segue he are closed, the contents of the memory cell will remain unmodified between one time-step and the next gradients gating structure allows information to be retained across many time-steps, and consequently also allows group that to flow across many time-steps. This allows the LSTM model to overcome the vanishing gradient properly occurs with most Recurrent Neural Network models. The basic workflow of a Long Short Term Memory Network is similar to the workflow of a Recurrent Neural Network with the only difference being that the Internal Cell State is also passed forward along with the Hidden State.

Long Short Term Memory Network consists of four different gates for different purposes as described below:-

- **Forget Gate(f)** : At forget gate the input is combined with the previous output to generate a fraction between 0 and 1, that determines how much of the previous state need to be preserved (or in other words, how much of the state should be forgotten). This output is then multiplied with the previous state
- **Input Gate(i)** : Input gate operates on the same signals as the forget gate, but here the objective is to decide which new information is going to enter the state of LSTM. The output of the input gate (again a fraction between 0 and 1) is multiplied with the output

of tan h block that produces the new values that must be added to previous state. This gated vector is then added to previous state to generate current state

- **Output Gate(o)** : At output gate, the input and previous state are gated as before to generate another scaling fraction that is combined with the output of tanh block that brings the current state. This output is then given out. The output and state are fed back into the LSTM block.

1.4 BIDIRECTIONAL - LONG SHORT TERM MEMORY (BI-LSTM)

Bidirectional recurrent neural networks (BRNN) connect two hidden layers of opposite directions to the same output. With this form of generative deep learning, the output layer can get information from past (backwards) and future (forward) states simultaneously. Invented in 1997 by Schuster and Paliwal, BRNNs were introduced to increase the amount of input information available to the network. For example, multilayer perceptron (MLPs) and time delay neural network (TDNNs) have limitations on the input data flexibility, as they require their input data to be fixed. Standard recurrent neural network (RNNs) also have restrictions as the future input information cannot be reached from the current state. On the contrary, BRNNs do not require their input data to be fixed. Moreover, their future input information is reachable from the current state.

BRNN are especially useful when the context of the input is needed. For example, in handwriting recognition, the performance can be enhanced by knowledge of the letters located before and after the current letter. The principle of BRNN is to split the neurons of a regular RNN into two directions, one for positive time direction (forward states), and another for negative time direction (backward states). Those two states' output are not connected to inputs of the opposite direction states. The general structure of RNN and BRNN can be depicted in the right diagram. By using two time directions, input information from the past and future of the current time frame can be used unlike standard RNN which requires the delays for including future information.

1.5 FBPROPHET

Forecasting at scale - Prophet is a forecasting procedure implemented in R and Python. It is fast and provides completely automated forecasts that can be tuned by hand by data scientists and analysts. Prophet is a procedure for forecasting time series data based on an additive model where non-linear trends are fit with yearly, weekly, and daily seasonality, plus holiday effects. It works best with time series that have strong seasonal effects and several seasons of historical data. Prophet is robust to missing data and shifts in the trend, and typically handles outliers well. Prophet is open source software released by Facebook's Core Data Science team.

FEATURES :

1. Accurate and fast -

Prophet is used in many applications across Facebook for producing reliable forecasts for planning and goal setting. It is found to perform better than any other approach in the majority of cases. We could forecast future trends and data by fitting the model in just a few seconds.

2. Fully automatic -

We are able to get a reasonable forecast on messy data with no manual effort. Prophet is robust to outliers, missing data, and dramatic changes in your time series.

3. Tunable forecasts -

The Prophet procedure includes many possibilities for users to tweak and adjust forecasts. Prophet allows to use human-interpretable parameters to improve your forecast by adding your domain knowledge.

4. Available in R or Python -

The Prophet procedure is implemented in R and Python, but they share the same underlying Stan code for fitting. Prophet allows to use whatever language we are comfortable with to get forecasts.

CHAPTER – 2

LITERATURE REVIEW

Abdul Quadir Md, Sanjit Kapoor, Chris Junni A.V, Arun Kumar Sivaraman, Kong Fah Tee, Sabireen H, Janakiraman N [1] has proposed the new approach for stock price prediction based on a Multi-Layer Sequential Long Short Term Memory (MLS LSTM) model that makes use of the Adam optimizer. Various algorithms like LSTM, Multi-layered LSTM along with various performance metrics like Normalised Root Mean Squared Error (RMSE), Mean Absolute Percentage Error measure (MAPE) and R^2 measure to analyse the efficiency of the trained models. The model is trained with stock data from the year 2017 till the year 2020. The dataset used is very less compared to the accuracy metrics obtained and MLS LSTM achieved a prediction accuracy of 95.9% and 98.1% on the testing data set. This MLS LSTM model is sensitive to overfitting and we must train the model with more data.

Xiao Teng, Xiang Zhang, Zhigang Luo[2] has proposed a novel multi-scale local cues and hierarchical attention-based LSTM (MLCA-LSTM) model to predict stock price trends by applying local descriptors. This model integrates local descriptors, multi-scale sliding windows, multi-branch LSTM modules and hierarchical attention mechanism altogether to boost stock price trend prediction performance. The model uses the approach of sliding window to predict the stock prices for the next forecasting days. In contrast with LSTM-based models, MLCA-LSTM degenerates more slowly than them. The reason is probably that the use of local descriptors reduces the length of original time series, which makes MLCA-LSTM capable of dealing with longer dependency relationship. The paper also gives conclusion that the number of the hidden states has a tremendous influence on the performance of both MLCA-LSTM and Adv-LSTM. MLCA-LSTM and Adv-LSTM tend to achieve better performance when the number of hidden states is set to 20 or 25. However, MLCA-LSTM yields better prediction performance than Adv-LSTM in most cases, and achieves the best result when the number of hidden states is set to 20.

Kelum Gajamannage, Yonggi Park, Dilhani I. Jayathilake [3] has proposed that classic linear models have no capability of capturing non-linear and non-stationary relationships in the data, nonlinear models might not cope with certain nonlinear and non-stationary signals as the underlying model does not fit with the data. The model is a many-to-one dual-LSTM with sequential data sampled using a moving window approach such that the succeeding window is shifted forward by one data instance from the preceding window. Sequential window training maximizes the performance of this dual-LSTM as it accelerates LSTMs' learning capability as well as it increases the LSTMs' robustness to new data. The performance analysis covers the dual-LSTM applied to nine time series obtained from three financial markets, stocks (Apple, Microsoft, Google), cryptocurrencies (Bitcoin, Ethereum, Cardano), and commodities (gold, crude oil, natural gas).

Pouya Khodaei, Akbar Esfahanipour, Hassan Mehtari Taheri [4] have proposed a CNN-LSTM-RESNET model which presented an innovative 3D-CNN for action detection, in which the features were extracted from the temporal and spatial dimensions. Piecewise Linear Regression (PLR) is a method to split the time series of close price into various segments using a predetermined Threshold. Image Segmentation is used to determine the objects and boundaries in a specific image. For the sake of clarity, a fitness function was used to obtain the Threshold of each stock's close price time series by considering its fluctuation. This model also developed a balancing algorithm by considering an over-sampling method, which considers the neighbors of a stock date. However, one of the drawbacks of their approach was creating unbalanced classes in which the number of holding days was far in excess of buying and selling points.

CHAPTER – 3

SYSTEM ANALYSIS

3.1 PROBLEM STATEMENT

The rate of investment and business opportunities in the Stock market can increase if an efficient algorithm could be devised to predict the short term price of an individual stock. Stock price forecasting is among one of the most challenging problems that financial institutions, businesses, and individual investors face. Many factors impact the validity of stock price forecasts, including economics, political contexts, and investor psychology. A company with healthy finances will perform better in the stock market than a company with less-than-ideal finances. The usage of machine learning paradigms in the financial sector, especially the stock market, has been on the rise for a long time[1]. Technical analysis is mainly based on a study of past and present stock prices to predict the future movements of stock prices. Stock market prediction is not linear, thus making it harder to predict a corporation's stock prices in a specific market. Our main objective is to forecast and predict the stock prices in order to make more informed and accurate investment decisions.

3.2 EXISTING SYSTEM

The existing systems available for stock price prediction and forecasting are trained with different models involving Machine Learning and Deep Learning[4]. Machine Learning Algorithms like Random Forest and Support Vector Machines are used which produce lesser accuracy. Deep Learning Algorithms like LSTM, ANN, RNN [3] models trained are prone to overfitting issues. The available models are also trained with static data using csv files which are downloaded and trained. This limits the dataset size available. Loss and Accuracy parameters also varies with different models and visualisation is not proper, precise and consistent. Some papers focus only on the limited attributes rather than all the important attributes needed for financial analysis.

3.2.1 LIMITATIONS OF EXISTING SYSTEM

1. Static datasets
2. Selection Bias
3. Incorrect Pre-processing
4. Overfitting issues
5. Lesser Accuracy
6. Higher Loss

3.3 PROPOSED SYSTEM

Our model is developed for the stock price prediction to help investors in making financial decisions. In most researches it focuses on "lowest price buy", "highest selling price". On the "lowest buy" and "highest selling" strategy of stocks occurs when stocks are at the lowest price and sell shares when prices are highest. For stock price predictions Bidirectional - LSTM technique is used with back propagation the dataset was pre-processed and tuned up for real analysis. Hence, we can upload stock price history i.e. open price, highest price, lowest price and close price of the day. Secondly, after pre-processing the data, our model analyses stock price history and gives input to the Back propagation algorithm. In addition, the proposed model also examines the use of the prediction system in real-world settings and issues associated with the accuracy of the overall values given. The back propagation gives output as final predicted rate comes. The proposed system can get the output graph of stock price prediction. The successful prediction of the stock will be a great asset for the stock market institutions and will provide real-life solutions to the problems that stock investors face. The backward propagation of errors, of back propagation, is a common method of training artificial neural networks.

3.3.1 ADVANTAGE OF THE PROPOSED SYSTEM

1. Improved Decision Making
2. Sliding Window Approach
3. Better Visualisations
4. Better Idea about Entry and Exit Points
5. Increased Accuracy
6. Reduced Bias
7. Less time complexity.
8. Less execution time.

3.4 REQUIREMENT ANALYSIS AND SPECIFICATION

3.4.1 REQUIREMENT DESCRIPTION

FRONT END : JUPYTER NOTEBOOK

The Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations and narrative text. Uses include: data cleaning and transformation, numerical simulation, statistical modelling, data visualization, machine learning, and much more. The software requirement specification is created at the end of the analysis task. The function and performance allocated to software as part of system engineering are developed by establishing a complete information report as functional representation, a representation of system behaviour, an indication of performance requirements and design constraints, appropriate validation criteria.

FEATURES OF JUPYTER NOTEBOOK

- In-browser editing for code, with automatic syntax highlighting, indentation, and tab completion/introspection.
- The ability to execute code from the browser, with the results of computations attached to the code which generated them.
- Displaying the result of computation using rich media representations, such as HTML, LaTeX, PNG, SVG, etc.
- For example, publication-quality figures rendered by the matplotlib library, can be included inline.
- In-browser editing for rich text using the Markdown markup language, which can provide commentary for the code, is not limited to plain text.
- The ability to easily include mathematical notation within markdown cells using LaTeX, and rendered natively by MathJax.

NOTEBOOK DOCUMENTS

Notebook documents contain the inputs and outputs of an interactive session as well as additional text that accompanies the code but is not meant for execution. In this way, notebook files can serve as a complete computational record of a session. interleaving executable code with explanatory text, mathematics, and rich representations of resulting objects. These documents are internally JSON files and are saved with the ipynb extension. Since JSON is a plain text format, they can be version-controlled and shared with colleagues. Notebooks may be exported to a range of static formats, including HTML (for example, for blog posts), reStructuredText, LaTeX, PDF, and slide shows, via the nbconvert command.

Furthermore, any ipynb notebook document available from a public URL can be shared via the Jupyter Notebook Viewer (nbviewer). This service loads the notebook document from the URL and renders it as a static web page. The results may thus be shared with a colleague, or as a public blog post, without other users needing to install the Jupyter notebook themselves.

FEATURES OF PYTHON

Python is a high-level general-purpose programming language whose design philosophy mostly emphasizes code readability with the use of significant code. Writing code in Python for both small and large scale projects becomes very easy. Python has a variety of features to offer to its users.

1. EASY TO CODE

Python is a very high-level programming language, yet it is effortless to learn. Anyone can learn to code in Python in just a few hours or a few days. Mastering Python and all its advanced concepts, packages and modules might take some more time. However, learning the basic Python syntax is very easy, as compared to other popular languages like C, C++, and Java.

2. EASY TO READ

Python code looks like simple English words. There is no use of semicolons or brackets, and the indentations define the code block. It can tell what the code is supposed to do simply by looking at it.

3. FREE AND OPEN-SOURCE

Python is developed under an OSI-approved open source license. Hence, it is completely free to use, even for commercial purposes. It doesn't cost anything to download Python or to include it in your application. It can also be freely modified and re-distributed. Python can be downloaded from the official Python website.

4. ROBUST STANDARD LIBRARY

Python has an extensive standard library available for anyone to use. This means that programmers don't have to write their code for every single thing unlike other programming languages.

There are libraries for image manipulation, databases, unit-testing, expressions and a lot of other functionalities. In addition to the standard library, there is also a growing collection of thousands of components, which are all available in the Python Package Index.

5. INTERPRETED

When a programming language is interpreted, it means that the source code is executed line by line, and not all at once. Programming languages such as C++ or Java are not interpreted, and hence need to be compiled first to run them. There is no need to compile Python because it is processed at runtime by the interpreter.

6. PORTABLE

Python is portable in the sense that the same code can be used on different machines. Suppose you write a Python code on a Mac. Suppose you want to run it on Windows or Linux later, no need to make any changes in it. As such, there is no need to write a program multiple times for several platforms.

7. OBJECT-ORIENTED AND PROCEDURE-ORIENTED

A programming language is object-oriented if it focuses design around data and objects, rather than functions and logic. On the contrary, a programming language is procedure-oriented if it focuses more on functions (code that can be reused). One of the critical Python features is that it supports both object-oriented and procedure-oriented programming.

8. EXTENSIBLE

A programming language is said to be extensible if it can be extended to other languages. Python code can also be written in other languages like C++, making it a highly extensible language.

9. EXPRESSIVE

Python needs to use only a few lines of code to perform complex tasks. For example, to display Hello World, simply need to type one line - `print("Hello World")`. Other languages like Java or C would take up multiple lines to execute this.

10. SUPPORT FOR GUI

One of the key aspects of any programming language is support for GUI or Graphical User Interface. A user can easily interact with the software using a GUI. Python offers various toolkits, such as Tkinter, wxPython and JPython, which allows for GUI's easy and fast development.

11. DYNAMICALLY TYPED

Many programming languages need to declare the type of the variable before runtime. With Python, the type of the variable can be decided during runtime. This makes Python a dynamically typed language.

12. HIGH-LEVEL LANGUAGE

Python is a high-level programming language because programmers don't need to remember the system architecture, nor do they have to manage the memory. This makes it super programmer-friendly and is one of the key features of Python.

13. SIMPLIFY COMPLEX SOFTWARE DEVELOPMENT

Python can be used to develop both desktop and web apps and complex scientific and numerical applications. Python's data analysis features help you create custom big data solutions without so much time and effort.

You can also use the Python data visualization libraries and APIs to present data in a more appealing way. Several advanced software developers use Python to accomplish high-end AI and natural language processing tasks.

14. OTHER ADVANCED PROGRAMMING FEATURES

Python contains several advanced programming features such as generators (used to create iterators with a different approach than most other languages) and list comprehensions (used to create new lists from other iterables). Python also has automatic memory management eliminating the need to manually allocate and free memory in the code.

ANACONDA

Anaconda Cloud is a package management service by Anaconda. Cloud makes it easy to find, access, store and share public notebooks, environments, and conda and PyPI packages. Cloud also makes it easy to stay current with updates made to the packages and environments you are using. Cloud hosts hundreds of useful Python packages, notebooks, projects and environments for a wide variety of applications.

We do not need to log in, or even to have a Cloud account, to search for public packages, download and install them authentication, tokens, upload, download, remove and search. Connect to and manage your Anaconda Cloud account. Upload packages you have created. Generate access tokens to allow access to private packages.

For developers, Cloud is designed to make software development, release and maintenance easy by providing broad package management support. Cloud allows for free public package hosting, as well as package channels, providing a flexible and scalable service for groups and organizations of all sizes.

APPLICATIONS PROVIDED IN ANACONDA DISTRIBUTION

The Anaconda distribution comes with the following applications along with Anaconda Navigator.

- JupyterLab
- Jupyter Notebook
- Qt Console
- Spyder
- Glueviz
- Orange3
- Visual Studio Code

JupyterLab: This is an extensible working environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.

Jupyter Notebook: This is a web-based, interactive computing notebook environment. It is possible to edit and run human-readable docs while describing the data analysis.

Qt Console: It is the PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting. graphical calltips and more.

Spyder: Spyder is a scientific Python Development Environment. It is a powerful Python IDE with advanced editing, interactive testing, debugging and introspection features.

VS Code: It is a streamlined code editor with support for development operations like debugging, task running and version control.

Glueviz: This is used for multidimensional data visualization across files. It explores relationships within and among related datasets.

Orange 3: It is a component-based data mining framework. This can be used for data visualization and data analysis. The workflows in Orange 3 are very interactive and provide a large toolbox.

3.4.2 HARDWARE SPECIFICATIONS

- Processor Type : Intel Core i5
- Speed : 3.40 GHz
- RAM : 4 GB
- Hard Disk : 500 GB
- Keyboard : 101/102 Standard Keys
- Mouse : Optical Mouse

3.4.3 SOFTWARE SPECIFICATIONS

- Operating System : Windows 10 / Windows 11
- Front End : Streamlit
- Coding Language : Python

CHAPTER – 4

SYSTEM DESIGN

4.1 SYSTEM DESIGN

4.1.1 INPUT DESIGN

Input design is the process of converting the user-originated input to a computer based format. The design decision for handling input specify how data are accepted for computer processing. Input design is a part of overall system design that needs careful attention.

The collection of input data is considered to be the most expensive part of the system design. Since the inputs have to be planned in such a way so as to get the relevant information, extreme care is taken to obtain the pertinent information. If the data going into the system is incorrect then the processing and outputs will magnify these errors. The goal of designing input data is to make data entry as easy, logical and free from errors as possible.

The following are the objectives of input design:

- To produce a cost effective method of input.
- To make the input forms understandable to the end users.
- To ensure the validation of data inputs.

The nature of input data is determined partially during logical system design. However, the nature of inputs is made more explicit during the physical design. The impact of inputs on the system is also determined. Effort has been made to ensure that input data remains accurate from the stage at which it is recorded and documented to the stage at which it is accepted by the computer. Validation procedures are also present to detect errors in data input, which is beyond control procedures. Validation procedures are designed to check each record, data item or field against certain criteria.

4.1.2 OUTPUT DESIGN

The output is designed in such a way that it is attractive, convenient and informative. Forms are designed in python with various features, which make the console output more pleasing. As the outputs are the most important sources of information to the users, better design should improve the system's relationships with us and also will help in decision-making. Form design elaborates the way output is presented and the layout available for capturing information.

In this proposed work, the output is designed in the form reports. The system is designed to generate various user friendly reports to help the business process. Figure 4.1 represent the different reports supported and Figure 4.2 represent the activity diagram :

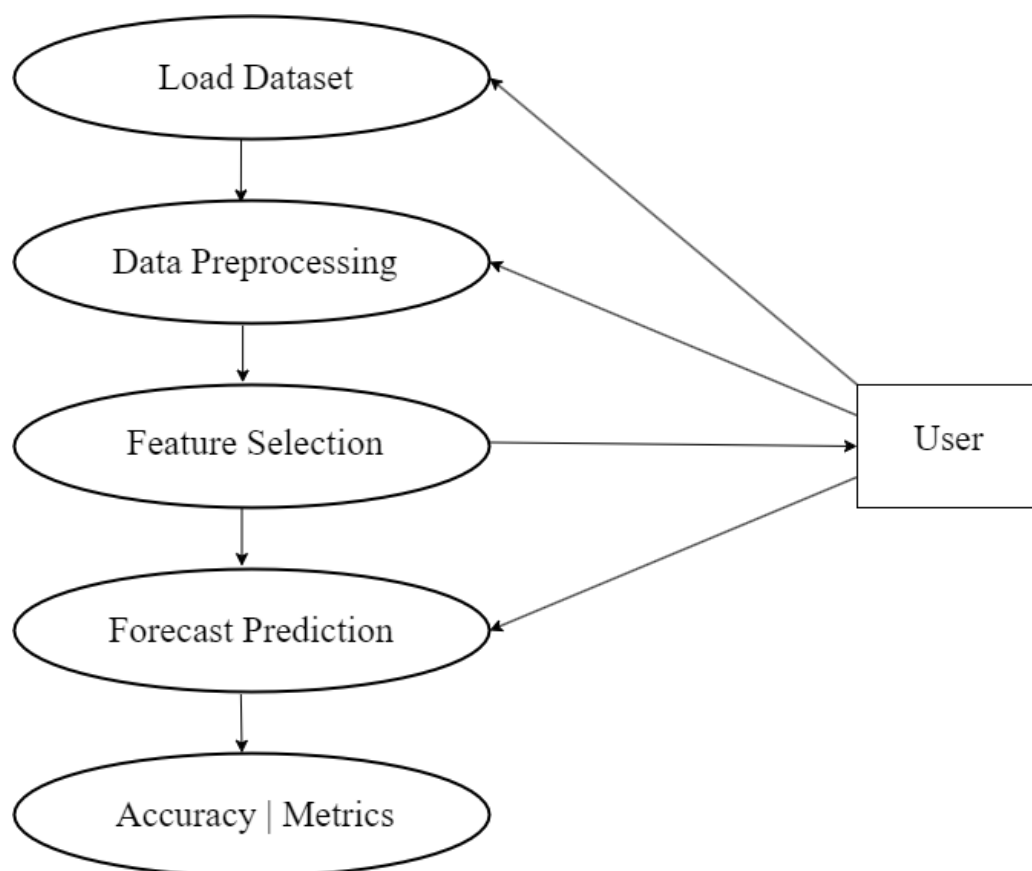


Fig 4.1 : USE CASE DIGAGRAM

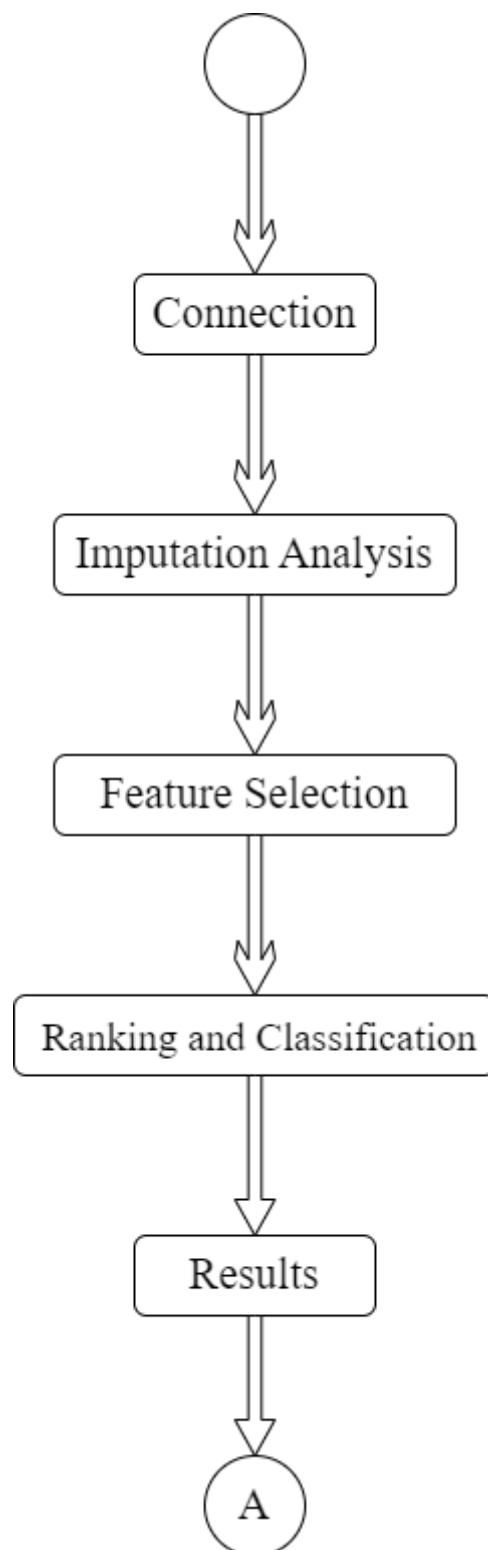


Fig 4.2 : ACTIVITY DIAGRAM

4.2 DESIGN CONSTRAINTS AND STANDARDS

The application considers two constraints namely, economic, ethical and social. Economic constraint is satisfied by providing plausible success rate and survey communications. This IEEE standard identifies and provides definitions for computed aided designs, character inference, computer assisted instruction, writer inference tasks, language processing, data processing and analysis among others. The system analyses dataset containing the day-wise actual past data and makes predictions for upcoming days using machine learning algorithms. Overall, we conclude that model predictions according to the current scenario are correct which may be helpful to understand the upcoming situation. The standard typically applies to automation tasks and learning techniques. These modules are implemented in the application subject to INSPEC Accession Number: 19660082.DOL: 10.1109/ACCESS 2020.2997311 IEEE Access

4.3 DESIGN ALTERNATIVES

The challenge of the stock price forecast is the most crucial component for companies and equity traders to predict future revenues. Predicting the market is challenging because the future is inherently unpredictable. Short-term traders are typically better served by waiting for confirmation that a reversal is at hand, rather than trying to predict a reversal will happen in the future. A successful and accurate prediction to the future stock prices ultimately results in profit maximisation. The challenges also includes that it is highly limited in its scope. Many predictors cannot be used, which is required to solve the stock price prediction problem. Various factors like demand and supply, economy, government policies, dividend declarations, political scenario also affect stock price prediction. To contribute to this aspect of information, numerous researchers are studying the different dimensions of applying machine learning algorithms to stabilise predictions.

4.4 MODULE DESCRIPTION

4.4.1 DETAILED DESIGN

Time series forecasting occurs when you make scientific predictions based on historical time stamped data. It involves building models through historical analysis and using them to make observations and drive future strategic decision-making. An important distinction in forecasting is that at the time of the work, the future outcome is completely unavailable and can only be estimated through careful analysis and evidence-based priors. Time series forecasting is the process of analysing time series data using statistics and modelling to make predictions and inform strategic decision-making. It's not always an exact prediction, and likelihood of forecasts can vary wildly—especially when dealing with the commonly fluctuating variables in time series data as well as factors outside our control. However, forecasting insight about which outcomes are more likely—or less likely—to occur than other potential outcomes. Often, the more comprehensive the data we have, the more accurate the forecasts can be.

The various Time series forecasting considerations are -

- Time horizons
- Dynamic and static states
- Data quality

4.5 DATA COLLECTION

The dataset includes stock information about a particular organization which is fetched using the Yahoo Finance open-source API available. The total size of the dataset varies according to the user who selects the number of years the user wants to analyse, train and predict.

4.6 DATA PRE-PROCESSING

The first step in creation of any Deep Learning Model is filtering the data which is to be given as input. Data pre-processing module involves encoding data into machine understandable format. It helps to clean, format and transform the raw data into efficient datasets, thereby making it ready for Machine Learning models. The steps involved are:

- Data and metadata are encoded from csv files into arrays using NumPy and pandas.
- The number of input variables are reduced which is known as dimensionality reduction.
- Data is scaled because scaling is a technique to make points in the data closer to each other.
- The various pre-processing techniques are applied to the dataset and passed to the models.
- The model is trained and predicts the future forecasted values for the particular stock,

4.7 TRAINING THE MODEL

- i. In the form of NumPy arrays the data and target data created is loaded.
- ii. Keras :
 - Shaping the data: Data in the form of NumPy array (data and target data) are converted to matrix form which is called shaping the data.
 - Weight is assigned to all data.
 - Different keras layers are assigned to the models according to their types.
 - Trainable and non-trainable parameters are given by the output layer.
- iii. Scikitlearn and tensorflow :
 - Using the tensorflow and scikitlearn python library data is split into training and Testing data.
- iv. Matplotlib :
 - To evaluate the model graphs are plotted which show training and validation loss.
- v. Finally, the model is evaluated and its prediction accuracy is calculated.

4.8 WEB APPLICATION

The backend is implemented in Python and the front end is implemented using Streamlit which uses python language. In python, the application links all the above-mentioned modules using their respective functions so that they are executed in a sequential manner:

- i. Data Pre-Processing
- ii. Index
- iii. Predict

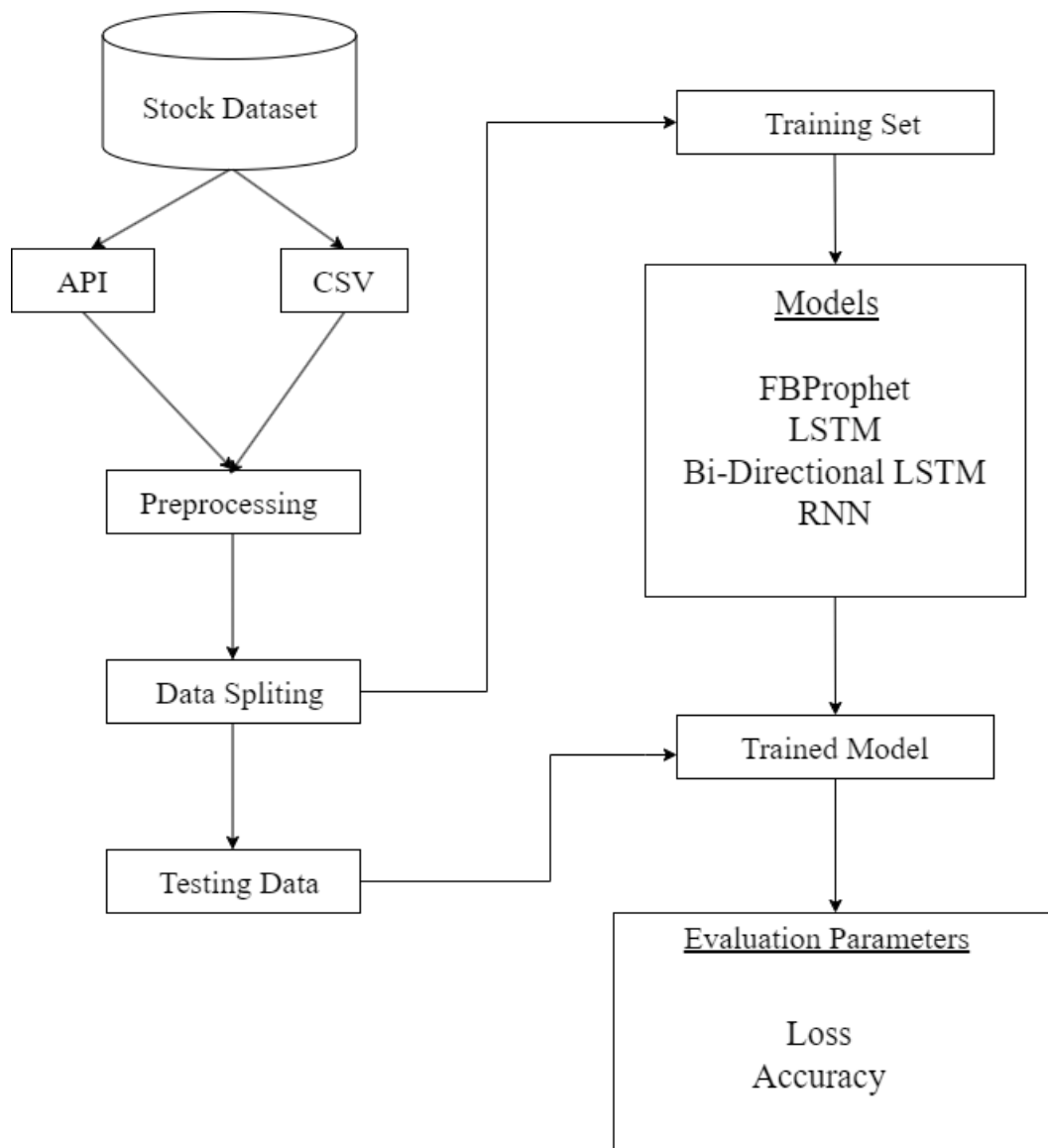


Fig 4.3 : PROPOSED WORKFLOW

Figure 4.3 represents the proposed workflow of the project. We pre-process the data and split the data into training and testing and train the model. The model is evaluated with the testing data and various evaluation parameters are obtained.

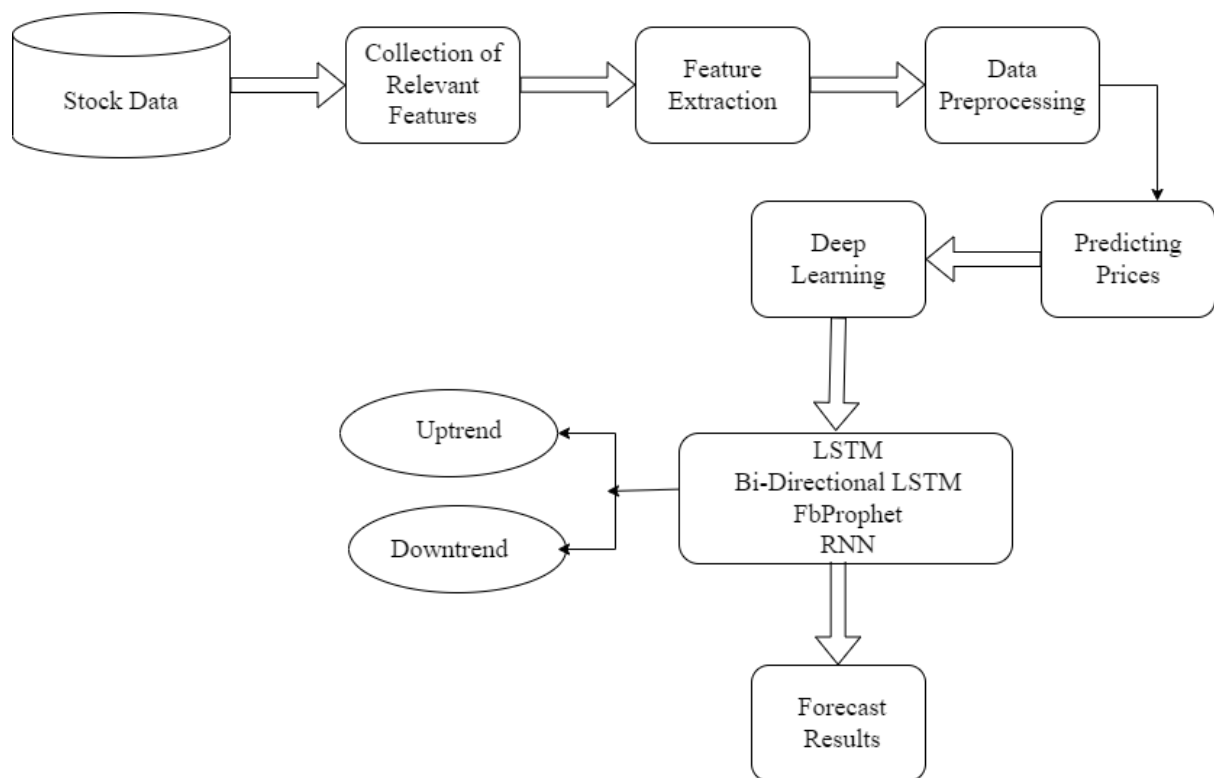


Fig 4.4 : OVERALL FLOW DIAGRAM

Figure 4.4 represents the overall flow diagram of the proposed model. We develop a system for the prediction and forecasting of stock prices using various Deep Learning and Forecasting Algorithms. The dataset used for the study contains the price and other necessary information about the particular stock. The visualisations in the graphs indicate the future trends and prices of the particular stock forecasted from the trained model.

CHAPTER – 5

IMPLEMENTATION

5.1 FRONT END

- Streamlit

5.1.1 STREAMLIT

Streamlit is an open-source Python library used for building web applications and data-driven applications. It is designed to make it easy to build and deploy apps that can be interacted with in real-time. Streamlit is specifically focused on data science and machine learning workflows, making it a popular tool for data scientists and developers. Streamlit allows users to create interactive web applications quickly and easily, without the need for extensive web development skills. The library is built on top of Python, and provides a simple and intuitive API for creating interactive dashboards, visualizations, and other applications.

One of the main advantages of Streamlit is its simplicity. It uses a single-file approach, where the entire application is contained in a single Python script, making it easy to deploy and share with others. It also provides a number of built-in widgets and components that allow users to interact with data and models in real-time. Streamlit also supports a variety of data visualization libraries, such as Matplotlib, Plotly, and Altair, making it easy to create rich and interactive visualizations for your data. Additionally, it provides support for machine learning libraries such as TensorFlow, PyTorch, and Scikit-learn, allowing you to easily build and deploy machine learning models within your application.

FEATURES

- Easy to use
- Fast prototyping
- Interactive widgets
- Automatic updates
- Easy deployment

5.2 BACK END

- Python

5.2.1 PYTHON

Python is a high-level, interpreted programming language that was first released in 1991. It is known for its simplicity, readability, and ease of use, which has made it popular for both beginners and experienced developers. One of the key features of Python is its strong support for object-oriented programming, which allows developers to write reusable and modular code. Python also has a large standard library and a wide range of third-party libraries, which can be easily installed and used to extend its functionality. Python is widely used in data science and machine learning due to its powerful libraries such as NumPy, Pandas, Scikit-Learn, and TensorFlow. These libraries provide the necessary tools for data manipulation, visualization, and analysis, making it a popular choice for scientific computing.

Python is also used in web development, scripting, automation, and many other domains. The syntax of Python is clean and easy to read, which makes it easier for developers to write and maintain code. Python supports various programming paradigms, including functional programming, procedural programming, and object-oriented programming. It also supports dynamic typing, which means that variables can change their data type during runtime.

FEATURES

- Simple and easy to learn
- Interpreted language
- Object-oriented
- Extensive libraries
- Cross-platform
- Dynamic typing

5.3 RESULTS

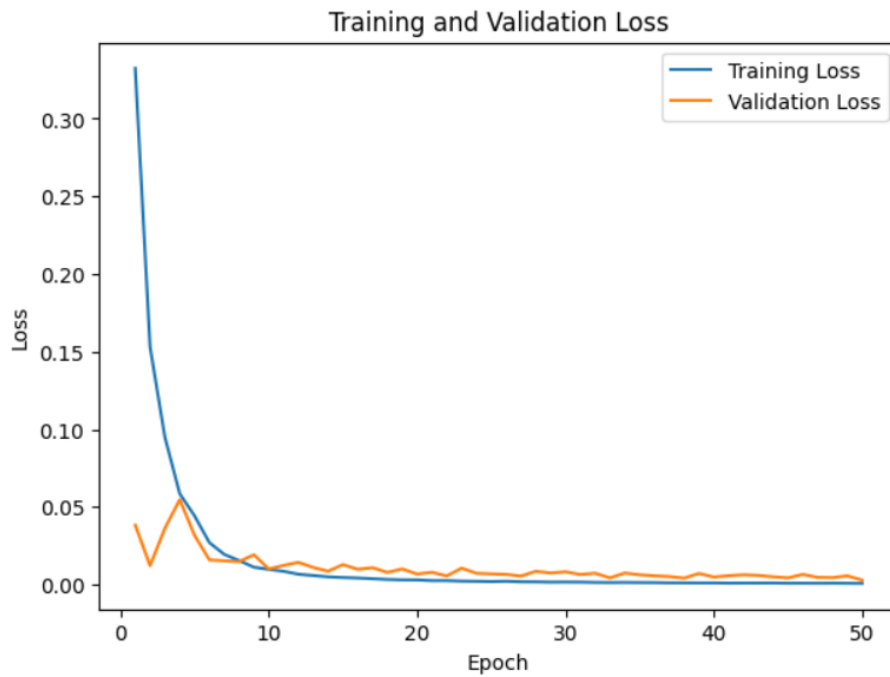


Fig 5.1 : LOSS GRAPH OF RNN MODEL



Fig 5.2 : LOSS GRAPH OF LSTM MODEL

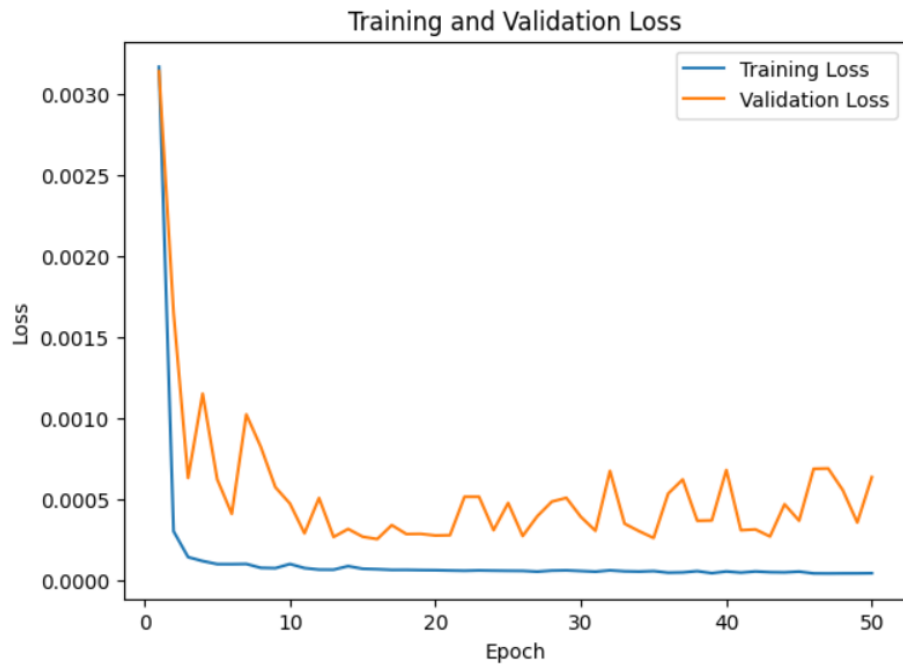


Fig 5.3 : LOSS GRAPH OF BI-LSTM MODEL

	Name	Loss %
0	RNN	0.285000
1	Bi-LSTM	0.063960
2	LSTM	0.088888

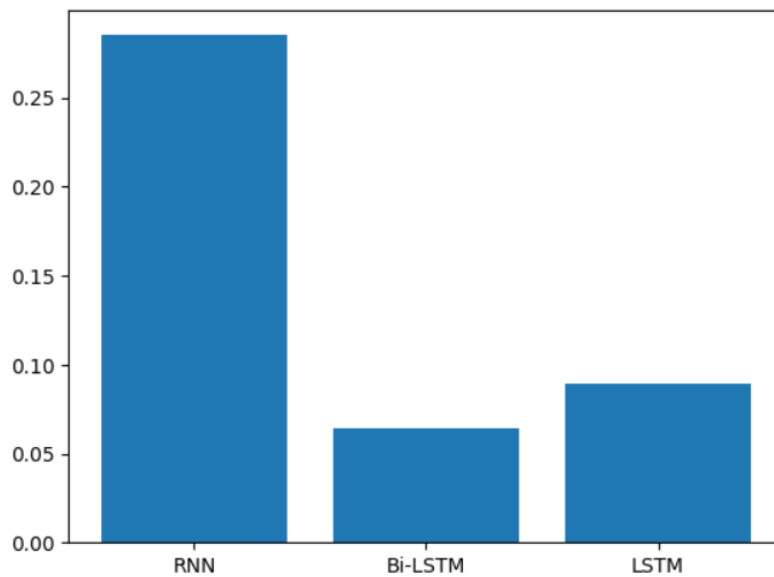


Fig 5.4 : COMPARISON OF LOSS OF DIFFERENT MODELS

MODEL	EPOCHS	LOSS %
RNN	50	0.285
LSTM	50	0.063
BI-LSTM	50	0.088

Table 5.1 : MODEL METRICS

Table 5.1 contains the loss metrics of the models trained and evaluated.

CHAPTER - 6

TESTING

6.1 INTRODUCTION

Software testing is a critical element of software quality assurance and represents the ultimate review of specification, design and coding. In fact, testing is the one step in the software engineering process that could be viewed as destructive rather than constructive. A strategy for software testing integrates software test case design methods into a well-planned series of steps that result in the successful construction of software. Testing is the set of activities that can be planned in advance and conducted systematically. The underlying motivation of program testing is to affirm software quality with methods that can economically and effectively apply to both strategic to both large and small-scale systems.

6.2 STRATEGIC APPROACH TO SOFTWARE TESTING

The software engineering process can be viewed as a spiral. Initially system engineering defines the role of software and leads to software requirement analysis where the information domain, functions, behaviour, performance, constraints and validation criteria for software are established. Moving inward along the spiral, we come to design and finally to coding. To develop computer software we spiral in along streamlines that decrease the level of abstraction on each turn. A strategy for software testing may also be viewed in the context of the spiral. Unit testing begins at the vertex of the spiral and concentrates on each unit of the software as implemented in source code. Talking another turn on outward on the spiral we encounter validation testing where requirements established as part of software requirements analysis are validated against the software that has been constructed. Finally we arrive at system testing, where the software and other system elements are tested as a whole.

6.3 UNIT TESTING

Unit testing focuses verification effort on the smallest unit of software design, the module. The unit testing we have is white box oriented and some modules the steps are conducted in parallel.

6.4 WHITE BOX TESTING

To follow the concept of white box testing we have tested each form. We have created independently to verify that Data flow is correct and all conditions are exercised to check their validity.

- All loops are executed on their boundaries. This type of testing ensures that all independent paths have been exercised at least once
- All logical decisions have been exercised on their true and false sides
- All loops are executed at their boundaries and within their operational bounds
- All internal data structures have been exercised to assure their validity paths.

6.5 CONDITIONAL TESTING

In this part of the testing each of the conditions were tested to both true and false aspects. And all the resulting paths were tested so that each path that may generate on a particular condition is traced to uncover any possible errors.

This type of testing selects the path of the program according to the location of definition and use of variables. This kind of testing was used only when some local variable were declared. The definition-use chain method was used in this type of testing.

6.6 DATA FLOW TESTING

This type of testing selects the path of the program according to the location of definition and use of variables. This kind of testing was used only when some local variable were declared. The definition-use chain method was used in this type of testing.

6.7 LOOP TESTING

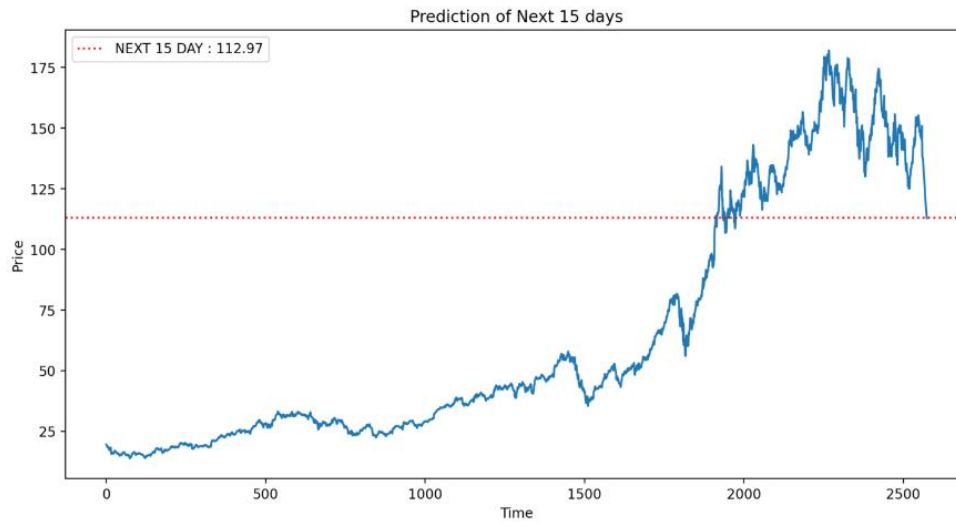
In this type of testing all the loops are tested to all the limits possible. The following exercise was adopted for all loops:

- All the loops were tested at their limits, just above them and just below them. All the loops were skipped at least once.
- For nested loops test the inner most loop first and then work outwards. For concatenated loops the values of dependent loops were set with the help of connected loop.
- Unstructured loops were resolved into nested loops or concatenated loops and tested as above.
- Each unit has been separately tested by the development team itself and all the input have been validated.

6.8 TEST CASE

A test case, in software engineering, is a set of conditions or variables under which a tester will determine whether an application, software system or one of its features is working as it was originally established for it to do. The mechanism for determining whether a software program or system has passed or failed such a test is known as a test oracle. In some settings, an oracle could be a requirement or use case, while in others it could be a heuristic. It may take many test cases to determine that a software program or system is considered sufficiently scrutinized to be released. Test cases are often referred to as test scripts, particularly when written - when they are usually collected into test suites.

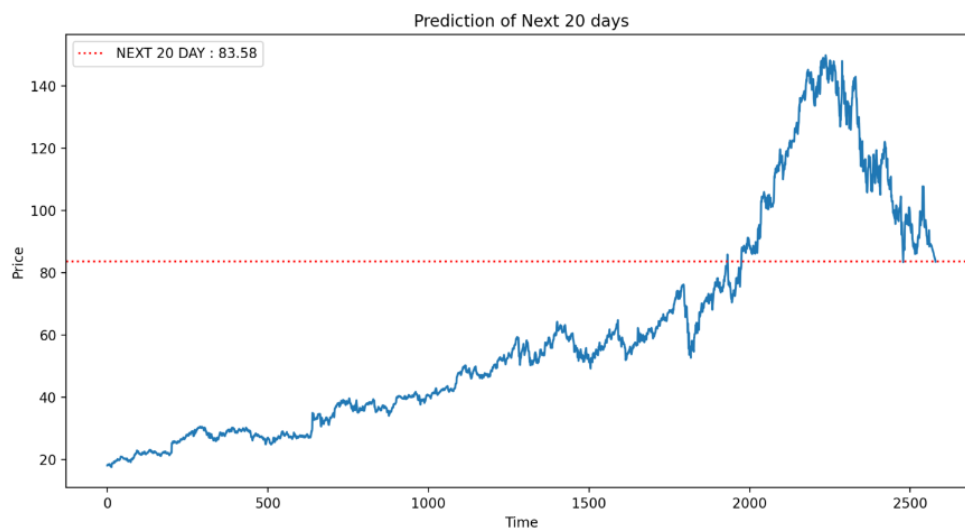
6.8.1 TEST CASE 1



INPUT : AAPL Stock data from 2013 till 2022

OUTPUT : Forecasted Price for the next 15 days

6.8.2 TEST CASE 2



INPUT : GOOGL Stock data from 2013 till 2022

OUTPUT : Forecasted Price for the next 20 days

CHAPTER - 7

CONCLUSION AND FUTURE WORK

The project proposes a new approach for stock price prediction based on a different models like LSTM, Bidirectional-LSTM and RNN. Bidirectional - LSTM is a better approach also because more layers in the model reduce the generality of the results. In an effort to build a more efficient and reliable prediction system in the future, fundamental analysis and sentiment analysis will be included in the prediction process. By doing this, we will be able to gauge the general sentiment of the public regarding the company, which will allow us to tweak the prediction technique based on the sentiment and make even more accurate predictions in the future. Changing the way the missing data is handled, that is, replacing null values with running averages or median of a set range of data can also help increase training accuracy. An effort to reduce the training time as well as resource requirements of LSTM must be made. The problem of overfitting must also be taken care of, as LSTM can be sensitive to it.

Future scope of this project will involve adding more parameters and factors like the financial ratios, multiple instances, etc. The more the parameters are taken into account more will be the accuracy. The algorithms can also be applied for analyzing the contents of public comments and thus determine patterns/relationships between the customer and the corporate employee. The use of traditional algorithms and data mining techniques can also help predict the corporation performance structure as a whole. In the future, we plan to integrate neural network with some other techniques such as genetic algorithm or fuzzy logic. Genetic algorithm can be used to identify optimal network architecture and training parameters. Fuzzy logic provides the ability to account for some uncertainty produced by the neural network predictions. This type of logic in conjunction with neural network could provide an improvement for stock market prediction.

REFERENCES

- [1] Novel Optimization Approach For Stock Price Forecasting Using Multi-Layered Sequential LSTM. *Applied Soft Computing* 134 (2023) 109830
- [2] Multi-scale local cues and hierarchical attention-based LSTM for stock price trend prediction. *Neurocomputing* 505 (2022) 92–100
- [3] Real-time forecasting of time series in financial markets using sequentially trained dual-LSTMs. *Expert Systems With Applications* 223 (2023) 119879
- [4] Forecasting Turning Points In Stock Price By Applying A Novel Hybrid CNN-LSTM-RESNET Model Fed By 2d Segmented Images. *Engineering Applications of Artificial Intelligence* 116 (2022) 105464
- [4] N.I. Sapankevych, R. Sankar, Time series prediction using support vector machines: a survey, *IEEE Comput. Intell. Mag.* 4 (2) (2009) 24–38.
- [5] P. Mondal, L. Shit, S. Goswami, Study of effectiveness of time series modeling (ARIMA) in forecasting stock prices, *Int. J. Comput. Sci. Eng. Appl.* 4 (2) (2014) 13.
- [6] Y. Chen, Z. Wei, X. Huang, Incorporating corporation relationship via graph convolutional neural networks for stock price prediction, in: *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*, 2018, pp. 1655–1658.
- [7] Y.T. Chen, W.N. Lai, E.W. Sun, Jump detection and noise separation by a singular wavelet method for predictive analytics of high-frequency data, *Comput. Econ.* 54 (2) (2019) 809–844.
- [8] N. Dalal, B. Triggs, Histograms of oriented gradients for human detection, in: *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05)* 1, 2005, pp. 886–893.
- [9] G.J. Deboeck, Trading on the edge: neural, genetic, and fuzzy systems for chaotic financial markets 39 (1994).
- [10] F. Feng, H. Chen, X. He, J. Ding, M. Sun, T.S. Chua, Enhancing stock movement prediction with adversarial training, *IJCAI* (2019)
- [11] K. Chen, Y. Zhou, F. Dai, A LSTM-based method for stock returns prediction: A case study of China stock market, in: *2015 IEEE International Conference on Big Data, big data, IEEE*, 2015, pp. 2823–2824.
- [12] F. Jiang, G. Tang, G. Zhou, Firm characteristics and Chinese stocks, *J. Manage. Sci. Eng.* 3 (4) (2018) 259–283.
- [13] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, *J. Mach. Learn. Res.* 13 (2) (2012).
- [14] A. Yadav, C.K. Jha, A. Sharan, Optimizing LSTM for time series prediction in Indian stock market, *Procedia Comput. Sci.* 167 (2020) 2091–2100.

- [15] J. Zhao, L. Itti, Classifying time series using local descriptors with hybrid sampling, *IEEE Trans. Knowl. Data Eng.* 28 (3) (2015) 623–637.
- [16] J. Zhao, L. Itti, shapedtw: Shape dynamic time warping, *Pattern Recogn.* 74 (2018) 171–184.
- [17] Y. Zhao, Y. Shen, Y. Zhu, J. Yao, Forecasting wavelet transformed time series with attentive neural networks, in: 2018 IEEE International Conference on Data Mining (ICDM), 2018, pp. 1452–1457.
- [18] X. Zhong, D. Enke, Predicting the daily return direction of the stock market using hybrid machine learning algorithms, *Financial Innov.* 5 (1) (2019) 4.
- [19] Y. Zhu, G. Zhou, Technical analysis: An asset allocation perspective on the use of moving averages, *J. Financ. Econ.* 92 (3) (2009) 519–544
- [20] K.j. Kim, Financial time series forecasting using support vector machines, *Neurocomputing* 55(1-2) (2003) 307–319

APPENDICES

APPENDIX - I

FRONT END CODE

```
import streamlit as st

from pandas_datareader import data as pdr

import yfinance as yfin

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import plotly.graph_objects as go

from plotly.subplots import make_subplots

from prophet import Prophet

from prophet.plot import plot_plotly

import datetime

from sklearn.preprocessing import MinMaxScaler

from keras.models import load_model

yfin.pdr_override()

with st.sidebar:

    choice = st.selectbox("Select the page",("Home","Analyse","Predict","FBPROPHET"))

if choice == "Home":

    st.header("Stock Price Prediction and Forecasting Using Machine Learning")
```

```
st.write("- Stock Price Prediction using machine learning helps you discover the future value  
of company stock and other financial assets traded on an exchange.")
```

```
st.write("- The entire idea of predicting stock prices is to gain significant profits. Predicting  
how the stock market will perform is a hard task to do.")
```

```
st.write("- There are other factors involved in the prediction, such as physical and  
psychological factors, rational and irrational behavior, and so on.")
```

```
st.write("- All these factors combine to make share prices dynamic and volatile.")
```

```
st.write("- This makes it very difficult to predict stock prices with high accuracy.")
```

BACK END CODE

```
if choice == "Analyse":
```

```
    st.title("Stock Chart")
```

```
    user_input = st.text_input("Enter Stock Ticker", "AAPL")
```

```
    start = st.date_input("Enter Start Date", datetime.date(2023, 1, 1))
```

```
    end = st.date_input("Enter End Date")
```

```
    df = pdr.get_data_yahoo(user_input, start, end)
```

```
    df_time = df.reset_index()
```

```
    data = pd.DataFrame(df_time)
```

```
    submit = st.button("Submit")
```

```
    if submit:
```

```
        st.subheader("Stock Summary")
```

```
        st.table(data.describe())
```

```
        fig = make_subplots(rows=2, cols=1, shared_xaxes=True)
```

```
        fig.add_trace(go.Candlestick(x=data['Date'], open=data['Open'], high=data['High'],
```

```
                                low=data['Low'], close=data['Close'], name='AAPL'), row=1, col=1)
```

```

fig.add_trace(go.Bar(x=data["Date"],y=data["Volume"],name="Volume"),row=2,col=1)

fig.layout.update(title_text=f'Analysing Price and Volume of {user_input}',
xaxis_rangeslider_visible=True)

fig.update_layout(autosize=False, width=800, height=800)

st.plotly_chart(fig)

elif choice == "FBPROPHET":

    st.title("Time Series Forecasting using FBProphet")

    start = st.date_input("Enter Start Date", datetime.date(2023, 1, 1))

    end = st.date_input("Enter End Date")

    user_input = st.text_input("Enter Stock Ticker", "AAPL")

    years = st.slider("Number of Future Years to Forecast", 1, 5)

    period = years * 365

    yfin.pdr_override()

    df = pdr.get_data_yahoo(user_input, start, end)

    df_time = df.reset_index()

    data = pd.DataFrame(df_time)

    DFTrain = data[['Date', 'Close']]

    DFTrain = DFTrain.rename(columns = {"Date": "ds", "Close": "y"})

    submit = st.button("Submit")

    if submit :

        model = Prophet()

        model.fit(DFTrain)

        future = model.make_future_dataframe(periods = period)

        forecast = model.predict(future)

```

```

st.subheader('Forecasted Prices')

forecastFig = model.plot(forecast)

ax = forecastFig.gca()

ax.set_xlabel("Time", size=10)

ax.set_ylabel("Stock Price ($)", size=10)

ax.set_facecolor((0.08, 0.08, 0.1))

forecastFig.patch.set_facecolor((0.13, 0.18, 0.25))

ax.xaxis.label.set_color('white')

ax.tick_params(axis='x', colors='white')

ax.yaxis.label.set_color('white')

ax.tick_params(axis='y', colors='white')

st.pyplot(forecastFig)

elif choice == "Predict":

    st.title("Stock Trend Forecasting")

    choice = st.selectbox("Select the algorithm", ("LSTM", "BI-LSTM", "RNN"))

    model = load_model(f'{choice}.h5')

    start = "2013-01-01"

    end = "2023-03-04"

    user_input = st.text_input("Enter Stock Ticker", "AAPL")

    df = pdr.get_data_yahoo(user_input, start, end)

    n_days = st.slider('Forecast Days', 0, 45, 15)

    submit = st.button("Submit")

    if submit :

        ds = df['Close']

```

```

normalizer = MinMaxScaler(feature_range=(0,1))

ds_scaled = normalizer.fit_transform(np.array(ds).reshape(-1,1))

train_size = int(len(ds_scaled)*0.70)

test_size = len(ds_scaled) - train_size

ds_train, ds_test = ds_scaled[0:train_size,:], ds_scaled[train_size:len(ds_scaled),:]

def create_ds(dataset,step):

    Xtrain, Ytrain = [], []

    for i in range(len(dataset)-step-1):

        a = dataset[i:(i+step), 0]

        Xtrain.append(a)

        Ytrain.append(dataset[i + step, 0])

    return np.array(Xtrain), np.array(Ytrain)

X_train, y_train = create_ds(ds_train,100)

X_test, y_test = create_ds(ds_test,100)

X_train = X_train.reshape(X_train.shape[0],X_train.shape[1] , 1)

X_test = X_test.reshape(X_test.shape[0],X_test.shape[1] , 1)

train_predict = model.predict(X_train)

test_predict = model.predict(X_test)

y_predicted = model.predict(X_test)

train_predict = normalizer.inverse_transform(train_predict)

test_predict = normalizer.inverse_transform(test_predict)

test = np.vstack((train_predict,test_predict))

st.subheader("Scaling the data")

fig2 = plt.figure(figsize = (12,6))

```

```

plt.plot(normalizer.inverse_transform(ds_scaled))

plt.plot(test)

st.pyplot(fig2)

st.subheader("Predictions vs Original")

fig2 = plt.figure(figsize = (12,6))

plt.plot(y_test, 'b', label = 'Original Price')

plt.plot(y_predicted, 'r', label = 'Predicted Price')

plt.xlabel('Time')

plt.ylabel('Price')

plt.legend()

st.pyplot(fig2)

fut_inp = ds_test[len(ds_test)-100:]

fut_inp = fut_inp.reshape(1,-1)

tmp_inp = list(fut_inp)

tmp_inp = tmp_inp[0].tolist()

lst_output=[]

n_steps=100

i=0

while(i < n_days):

    if(len(tmp_inp)>100):

        fut_inp = np.array(tmp_inp[1:])

        fut_inp=fut_inp.reshape(1,-1)

        fut_inp = fut_inp.reshape((1, n_steps, 1))

        yhat = model.predict(fut_inp, verbose=0)

```

```

    tmp_inp.extend(yhat[0].tolist())

    tmp_inp = tmp_inp[1:]

    lst_output.extend(yhat.tolist())

    i=i+1

else:

    fut_inp = fut_inp.reshape((1, n_steps,1))

    yhat = model.predict(fut_inp, verbose=0)

    tmp_inp.extend(yhat[0].tolist())

    lst_output.extend(yhat.tolist())

    i=i+1

fig2 = plt.figure(figsize = (12,6))

plot_new=np.arange(1,101)

plot_pred=np.arange(101,101+n_days)

plt.plot(plot_new, normalizer.inverse_transform(ds_scaled[len(ds_scaled)-100:]))

plt.plot(plot_pred, normalizer.inverse_transform(lst_output))

st.pyplot(fig2)

ds_new = ds_scaled.tolist()

ds_new.extend(lst_output)

final_graph = normalizer.inverse_transform(ds_new).tolist()

fig2 = plt.figure(figsize = (12,6))

plt.plot(final_graph,)

plt.ylabel("Price")

plt.xlabel("Time")

plt.title(f"Prediction of Next {n_days} days")

```

```

plt.axhline(y=final_graph[len(final_graph)-1], color = 'red', linestyle = ':', label = 'NEXT
{0} DAY : {1}'.format(n_days,round(float(*final_graph[len(final_graph)-1]),2)))

plt.legend()

st.pyplot(fig2)

ranger = []

for i in range(0,n_days):

    ranger.append(i)

price = final_graph[len(final_graph)-n_days:]

unpacked_list = [item for sublist in price for item in sublist]

df = pd.DataFrame({"Date":ranger, "Price":unpacked_list})

fig = go.Figure()

fig.add_trace(go.Scatter(x=df["Date"], y=df["Price"], name="forecast"))

fig.layout.update(title_text='Time Series data with Rangeslider',
xaxis_rangeslider_visible=True)

st.plotly_chart(fig)

unpacked_list = [item for sublist in final_graph for item in sublist]

x_date = []

for i in range(len(ds_new)):

    x_date.append(i)

df = pd.DataFrame({"Date":x_date, "Price":unpacked_list})

fig = go.Figure()

fig.add_trace(go.Scatter(x=df["Date"],y=df["Price"],name="forecast", marker=dict(b)))

fig.layout.update(title_text='Time Series data with Rangeslider',
xaxis_rangeslider_visible=True)

st.plotly_chart(fig)

```

APPENDIX – II

Stock Price Prediction and Forecasting Using Machine Learning

- Stock Price Prediction using machine learning helps you discover the future value of company stock and other financial assets traded on an exchange.
- The entire idea of predicting stock prices is to gain significant profits. Predicting how the stock market will perform is a hard task to do.
- There are other factors involved in the prediction, such as physical and psychological factors, rational and irrational behavior, and so on.
- All these factors combine to make share prices dynamic and volatile.
- This makes it very difficult to predict stock prices with high accuracy.

Fig 7.1 : HOME PAGE

Stock Trend Forecasting

Select the algorithm

LSTM

Enter Stock Ticker

AAPL

Forecast Days



Submit

Fig 7.2 : PREDICTION PAGE

Stock Chart

Enter Stock Ticker

Enter Start Date

Enter End Date

Fig 7.3 : ANALYSE PAGE

Time Series Forecasting using FBProphet

Enter Start Date

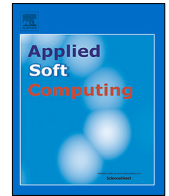
Enter End Date

Enter Stock Ticker

Number of Future Years to Forecast

1 5

Fig 7.4 : FBPROPHET PAGE



Novel optimization approach for stock price forecasting using multi-layered sequential LSTM

Abdul Quadir Md^a, Sanjit Kapoor^a, Chris Junni A.V.^a, Arun Kumar Sivaraman^b, Kong Fah Tee^{c,*}, Sabireen H.^a, Janakiraman N.^d

^a School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, 600127, India

^b Digital Engineering, Solution Center-H, Photo Inc. DLF Cyber City, Chennai, 600089, India

^c Faculty of Engineering and Quantity Surveying, INTI International University, 71800 Nilai, Malaysia

^d Department of Electronics and Communication Engineering, K.L.N. College of Engineering, Madurai, Tamil Nadu, India

ARTICLE INFO

Article history:

Received 3 July 2022

Received in revised form 1 November 2022

Accepted 10 November 2022

Available online 15 November 2022

Keywords:

Recurrent neural network

Forecasting

Stock market

Adam optimizer

Long short-term memory

ABSTRACT

Stock markets can often be one of the most volatile places to invest. Statistical analysis of past stock performance and external factors play a major role in the decision to buy or sell stocks. These factors are all used to maximize profits. Stock price index forecasting has been a subject of great research for many years, and several machine learning and deep learning algorithms have been proposed to simplify this complex task, but little success has been found so far. In order to forecast stocks accurately, it is crucial to understand the context-specific dependence of stock prices on their past values. The use of Long Short Term Memory (LSTM), which is capable of understanding long-term data dependencies, can help overcome this obstacle. In this context, this paper proposes a novel optimization approach for stock price prediction that is based on a Multi-Layer Sequential Long Short Term Memory (MLS LSTM) model which makes use of the adam optimizer. Additionally, the MLS LSTM algorithm uses normalized time series data divided into time steps to determine the relationship between past values and future values in order to make accurate predictions. Furthermore, it eliminates the vanishing gradient problem associated with simple recurrent neural networks. The stock price index is forecasted by taking into account past performance information along with past trends and patterns. The results illustrate that a 95.9% prediction accuracy is achieved on the training data set and a 98.1% accuracy on the testing data set with the MLS LSTM algorithm, which dramatically exceeds the performance of other machine learning and deep learning algorithms. The mean absolute percentage error was observed to be 1.79% on the training set and 2.18% on the testing set, respectively. Moreover, the proposed model is able to estimate the stock price with a normalized root mean squared error of 0.019, thus giving an accurate forecast and making it a feasible real-world solution.

© 2022 Published by Elsevier B.V.

1. Introduction

The price of a stock or a share on the stock market is the amount of money it costs to buy one share in a publicly-traded company. This price is rarely constant and fluctuates on a very regular basis. It mainly depends on the company's past performances but also depends on factors like the company's perceived reputation among other things. The demand and supply of the stock also heavily dictate the price of the stock at any given moment. Higher the demand, the higher the value and vice versa. The buying and selling of the stocks are done in the stock market, which allows the buyers and sellers to meet, interact and transact among themselves. Numerous researchers and analysts

have been attempting to accurately gauge and evaluate stocks to predict their future trends and value. The analysis that is performed is major of two types: Fundamental Analysis and Technical Analysis. The focus of fundamental analysis is on gathering and processing information from sources that include a company's financial records, economic reports, information about assets, market shares, etc. Focus is largely on the company's financial performance including profitability, liquidity, growth trajectory, etc. These factors are considered in determining how healthy the company is, which in turn gives the analysts an idea about whether or not the company will perform well in the future. A company with healthy finances will perform better in the stock market than a company with less-than-ideal finances. Technical analysis is mainly based on a study of past and present stock prices to predict the future movements of stock prices. Price, volume and supply-demand of the stock are taken into consideration.

* Corresponding author.

E-mail address: kongfah.tee@newinti.edu.my (K.F. Tee).

The usage of machine learning paradigms in the financial sector, especially the stock market, has been on the rise for a long time. Several algorithms, including Linear Regression, Support Vector Machines (SVM), Auto-Regressive Integrated Moving Average (ARIMA), etc. have been used with varying degrees of accuracy. One of the major obstacles in using simple machine learning paradigms such as regression for stock price prediction is that the prices of stocks can be highly random or can be represented as complex time series models. This means that the past data of the stock prices often dictates the future prices of the stock, which regression and SVM models do not take into account. This problem can be solved by the use of a deep learning paradigm that incorporates the historical price of the stock into the prediction of the future value, such as the LSTM technique. LSTM allows for the learning of long-term dependencies in the data to perform highly accurate predictions. In this study, the usage of the Multi-Layered Sequential LSTM technique for the prediction of stock prices is proposed. The LSTM technique is a kind of Recurrent Neural Network (RNN) which has the capacity to learn from and use past data to predict future outcomes. Conventional RNN is not able to handle long-term dependencies in data. There is also no fine control over what context in data is moved ahead and what amount of the past context needs to be forgotten by the network. Another huge drawback of conventional RNN is the problem of vanishing gradient. The gradient loss function of a neural network approaches zero as more layers using certain activation functions are added to it. This is known as a vanishing gradient and it makes the network hard to train. All of these shortcomings are overcome by LSTM networks. LSTM networks have the ability to selectively remember patterns for long sequences. This allows it to learn the dependency of the data on its past values and perform the prediction more accurately. LSTM models also allow fine control over selective forgetting of the past context of the data and also completely eliminate the problem of vanishing gradient. These advantages provided by the LSTM networks make it a suitable candidate for the prediction of future stock prices.

The remainder of the paper is arranged as follows: Section 2 outlines the work in the area of stock price prediction. Section 3 introduces the concept of LSTM and describes its function. Section 4 presents the proposed architecture and the methodology for the estimation of stock prices. In Section 5, the performance metrics and the experimental results are discussed and compared with similar work done with different techniques. The paper concludes with future directions in Section 6.

2. Related work

Several works that use Machine Learning techniques to predict stock prices exist in the public domain for us to review. In [1] the author has made use of a simple linear regression to forecast the price of the S&P index of 500. The data used in the predictive analysis consisted of 7 columns that include low, high, date, volume, close, adjusted close and open in total. The authors normalized the data first and then based on the number of features selected, built two linear regression models [2]. Model 1 made use of all 7 features of the dataset, whereas, in Model 2, the author omitted 2 features, namely Volume and Adjusted Close. Both models performed very well, with the first model having an R^2 of 0.997 and the second model having an R^2 score of 0.992. Despite both models having very good performance metrics, it must be noted that according to the author, S&P 500 data consists of the value of 500 stocks and it was chosen because it is sensitive and predictable. Therefore the model may not perform when the dataset is changed to something less predictable. The authors in [3] performed a predictive analysis of stock prices using Linear

Regression and Support Vector Machine (SVM) [4] models. The analysis was performed using R language with RStudio being the development environment. The authors used the Application Programming Interface (API) package provided by R to fetch the stock data of The Coca-Cola Company. The data was for exactly 1 year, ranging from January 2017 to 2018. The models were built using R's built-in `lm()` command for Linear Regression and the `svm()` command found in the `liquid SVM` package for SVM. The R^2 value of the Linear Regression model was observed to be 0.73 and that of the SVM model was 0.93. It shows that SVM performs significantly better than the LR model and has high accuracy.

In [5] the authors performed a comprehensive study of the usefulness of time series modelling (ARIMA) in stock forecasting. The data used in the study was that of fifty-six companies from seven different sectors and was obtained from the official website of the National Stock Exchange (NSE) India. Twenty-three months of training data from April 2012 to February 2014 was used for the prediction of the stock prices for the next month. ARIMA model was used in the predictive analysis. The results obtained showed that the accuracy of the ARIMA model in stock price prediction was above 85% for all the sectors. When looking at the sectors individually, it can be seen that the FMCG sector prediction had the highest accuracy but the Banking sector prediction accuracy was relatively lower. The work done in [6] helps in deciding which RNN (recurrent neural network) is best suited for the task. Three variants of RNN are tested. They are VRNN (Vanilla RNN), LSTM and Gated Recurrent Unit (GRU). GRUs, LSTM, RNN and CNN provide exceptional results in stock price forecasting as shown in [7–10]. A general rule of thumb is to use the simpler model as long as it does the job as simpler models perform well over time. As the name suggests VRNN is one of the most basic or naive RNNs. The major drawback of VRNN is that it suffers from the vanishing gradient problem, which in turn causes the gradient of the loss function to decay over time rendering the model ineffective. LSTM hopes to prevent loss function decay and is able to learn dependencies in long term. This makes LSTM an excellent option for time series analysis. GRU is LSTM but simpler and shows slightly better results when compared to LSTM but that is due to the fact that the dataset used is smaller and hence the potential of the long-term dependencies is shrouded. LSTM is chosen because of its ability to work well with large datasets.

In [11], the author uses Neural Network, LSTM to forecast the stock estimates. Stock prices of big corporations such as Apple, Google, etc. have been used for the prediction but the time frame of the stock price data is not mentioned explicitly. The author has reported an accuracy of 94% while using an Artificial Neural Network and an accuracy of 97% while using the LSTM algorithm. Although the reported accuracy for the models used is very high, the author has mentioned very little about the dataset used to perform the analysis and also the methodologies involved in building the model. The authors of [12] use LSTM to predict stock prices of two stocks on the NYSE with 15 years of stock prices taken from Yahoo Finance. The authors use different numbers of epochs for training to see at what number of epochs the LSTM successfully predicts the stock prices with decent accuracy. They have concluded that more epochs and less training data are the best way to go and also mention that the LSTM is a very capable model when it comes to predicting stock prices. In [13] the authors made use of four different techniques using deep learning for the forecast of prices of stocks from the New York and the National stock exchange. The authors trained LSTM, Multilayer Perceptron (MLP) [14], CNN and RNN using the stock prices of a company from the National Stock Exchange (NSE). The models were then used to predict the stock prices of five separate companies that were picked from the NSE and the NYSE. The authors compared their results with the ARIMA model

and showed that specialized neural network models outperform existing linear models like ARIMA. In [15], a comprehensive comparison between the performance of a LSTM and a MLP approach in the short-term prediction of stock prices is presented. The focus is on granular movements of the stock price and thus the data contains minute-by-minute stock prices for all trading days over the span of 1 year of 10 stocks that are listed on the NYSE.

In [16] the author performs an investigative study to analyse the effectiveness of networks of LSTMs for the purpose of predicting stock prices. The author tabulated the RMSE test error of LSTM networks with 50, 100, 250 and 500 hidden layers and proved that usage of LSTM networks is an effective method of predicting stock prices mainly because LSTM networks are resistant to overfitting and perform better when larger networks are joined together. The authors of [17] predict the prices of Chinese stocks with the help of LSTM. The authors claim that the reason behind choosing Chinese stocks [18] for prediction is that they are much more unpredictable. The data used for prediction contained 10 learning features and was transformed into 30-day-long sequences. According to the authors, the LSTM model used by them enhanced the accurate forecast of stock prices by 27.2% in contrast to the approach of random prediction. A comparison between grid and manual search techniques for initialization of hyper-parameters for neural networks was done in [19] and the authors concluded that the manual search technique is capable of finding as good or even better models than the grid search technique and in less time. The authors have suggested the use of a manual search technique for judging the hyper-parameters of neural network models. In [20] the authors perform experiments to find out the optimal number of hidden layers of a LSTM model that is used for stock price prediction. Stock price data from four companies — Maruti, ICICI, Reliance and TCS were used for training and prediction of the LSTM approach. The hidden layers of the LSTM models used in the experiment varied from one to seven. The authors observed that the LSTM model with $n = 1$ hidden layer performed the best, with a RMSE score of 5.85, while the RMSE score for $n = 2$ to $n = 7$ rose from 8.86 to 5.89, suggesting that one hidden layer must be used for stock price prediction as it yields higher accuracy.

Based on the literature survey, it can be concluded that deep learning paradigms perform better than simple machine learning paradigms, and thus are better suited for the task of stock price prediction. This can be attributed to the fact that deep learning systems can automatically learn complex mappings from inputs and outputs. The usage of deep learning algorithms such as RNN leads to the obstacle of vanishing gradient which also needs to be overcome. Thus, we can conclude that using LSTM is the best way to forecast stock prices, despite its drawbacks, which include high computational power and resource requirement along with huge training times. With time-series data, LSTM is very helpful in overcoming the problem of vanishing gradients using backpropagation. When combined with minmax normalization, LSTM is the best technique for accurately predicting stock prices while also avoiding any bias that may arise from outliers.

3. Long short-term memory

LSTM networks are considered to be the most effective solution to sequence prediction problems. The most advantageous property of LSTM is that it can remember patterns for a large number of sequences. This gives LSTM an advantage over standard feed-forward NN and RNN, which lack the ability to use past observations to predict future values. A typical LSTM network contains multiple memory blocks that are also called cells. Each cell transfers two states to its subsequent cell; the hidden and cell state as shown in Algorithm 1. The remembering of information

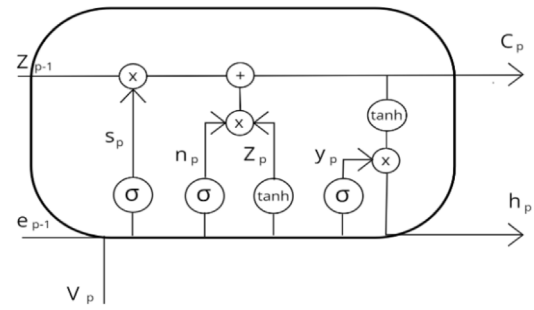


Fig. 1. LSTM cell.

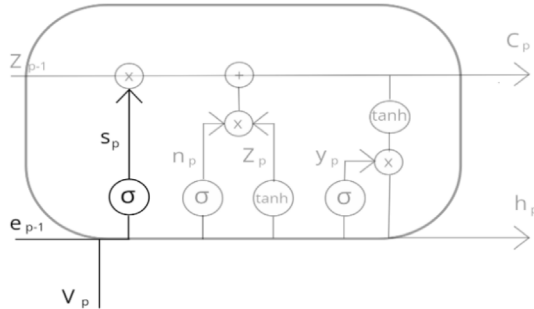


Fig. 2. Forget gate of LSTM cell.

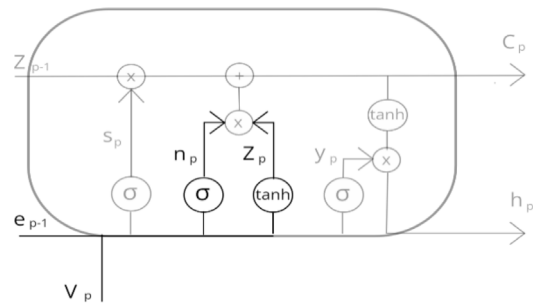


Fig. 3. Input gate of LSTM cell.

is done by the memory blocks. Manipulations to this memory are performed by three major structures that are known as gates. The illustrations of an LSTM cell along with every gate (namely, forget gate, input gate and output gate) are shown below in Figs. 1–4 and the roles played by them along with Eqs. (1)–(5) are discussed.

Forget Gate: This is the first gate in the LSTM cell and its function is to decide whether or not the information from the previous stamp is to be retained or not. The information from the present input V_p and hidden state e_{p-1} are taken and a sigmoid function is applied which makes its output between 0 and 1, after which it is multiplied by the cell state of the previous timestamp. If the final value is 0, then it forgets everything and if it is 1, then it forgets nothing. The weights are represented by D and bias is represented by g .

$$s_p = \sigma(D_s \times [e_{p-1}, V_p] + g_s) \quad (1)$$

Input Gate: In the input gate, another sigmoid function is applied to the current state V_p and hidden state e_{p-1} and a value between 0 and 1 is obtained, after which the tanh function is applied to it. It then takes the values of vector input n_p and carries out the addition in step-wise, after which, the cell state is updated

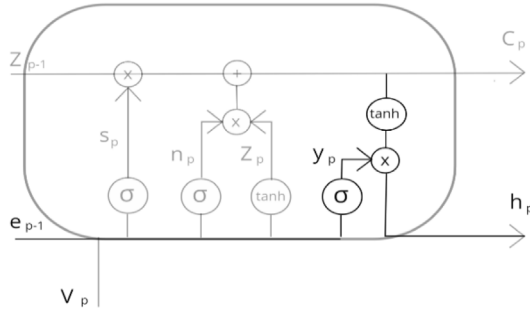


Fig. 4. Output gate of LSTM cell.

to a new cell state C.

$$n_p = \sigma(D_n \times [e_{p-1}, V_p] + g_n) \quad (2)$$

$$Z_p = \tanh(D_z \times [e_{p-1}, V_p] + g_z) \quad (3)$$

Output Gate: A third sigmoid function is applied to the current state and previous hidden state and the new cell state generated in the Input Gate is passed through the tanh function, both of these outputs are multiplied point by point and decide what information the hidden state for the next cycle will carry. The hidden state, along with the new cell state, is carried over to the next step.

$$y_p = \sigma(D_y \times [e_{p-1}, V_p] + g_y) \quad (4)$$

$$e_p = y_p \times \tanh(Z_p) \quad (5)$$

The abovementioned structure of the LSTM cell allows for backpropagation of weights and errors, which provides for fine-grained tuning of weights and biases, leading to greatly accurate predictions. It also helps in tackling the problems of vanishing and booming gradients.

Algorithm 1: LSTM cell computation algorithm

Input: V_p

Output: y_p

Initialize: $e_{p-1}, D_s, D_n, D_y, g_s, g_n, g_y$

while i less than time_steps **do**

Step 1: Forget gate of LSTM

$$s_p = \sigma(D_s \times [e_{p-1}, V_p] + g_s)$$

Step 2: Input gate of LSTM

$$n_p = \sigma(D_n \times [e_{p-1}, V_p] + g_n)$$

$$Z_p = \tanh(D_z \times [e_{p-1}, V_p] + g_z)$$

Step 3: Output gate of LSTM

$$e_p = y_p \times \tanh(Z_p)$$

$$y_p = \sigma(D_y \times [e_{p-1}, V_p] + g_y)$$

end while

4. Proposed methodology of multi-layer sequential long short-term memory

MLS LSTM consists of two major parts namely data preparation and sequential LSTM as shown in Fig. 5. The sequential LSTM consists of 4 layers out of which 3 layers are vanilla LSTM and the other layer is Dense. A high number of layers lead to better results but require greatly larger computational power, and thus a tradeoff between performance and resource use was reached with 4 layers. The close column is sliced from the main dataset and is used as the input for the MLS LSTM after being checked for any null values.

MLS LSTM much like any other LSTM, needs its data fed as time steps. Based on the empirical observations, a time step of 100 data points is utilized and a 65:35 train-to-test split is adopted. Mean Absolute Percentage Error (MAPE), Root Mean Squared Error (RMSE) and R-squared measures are used for evaluating MLS LSTM's performance. Algorithm 2 represents the construction of the MLS LSTM model.

4.1. Importing data

The required dataset is downloaded from Yahoo Finances. The dataset in use contains the stock information of Samsung dating from 2016 to 2021. The dataset contains the low, high, date, volume, close, adjusted close and open.

- The Date column contains the stock price date
- The Open column contains information about the price at which the stock opened on the day
- The High column represents the highest price the stock touched on the day
- The Low column gives the lowest price of the stock on the day
- The Close column gives information about the price at which the stock was closed
- Adj Close value adjusts for the corporate activities
- The volume column gives the number of shares of the companies that were transacted on the given day.

Of the abovementioned fields, the columns of interest are the Close column and the Date column. This is because date-wise training and prediction of the closing stock price are to be performed using the proposed MLS LSTM model.

4.2. Cleaning data

The first major step is to handle missing data. Each row of the dataset is scanned to check for missing values. It is observed that the columns Open, High, Low, Close, Adj and Volume have 5 NA values each. The NA values are filled using the fillna function of Pandas library. The number of missing values in each column is then visualized using the library 'missingno'. In the bar plot that is plotted in Fig. 6, it can be seen that the Date column has 1227 rows with proper values whereas all the other rows only have 1222 rows with proper values (because of the 5 NA values in each row).

Since the stock prices of stocks do not differ massively from one day to the next, the values from the previous rows can be used to fill the NA values without adding inaccuracies to the data. This method is preferred over other using mean or median values to replace the null values because it requires far less computational power while also being relatively accurate. For this purpose, the pad method of the fillna function is used to fill the NA values with the values from the previous rows.

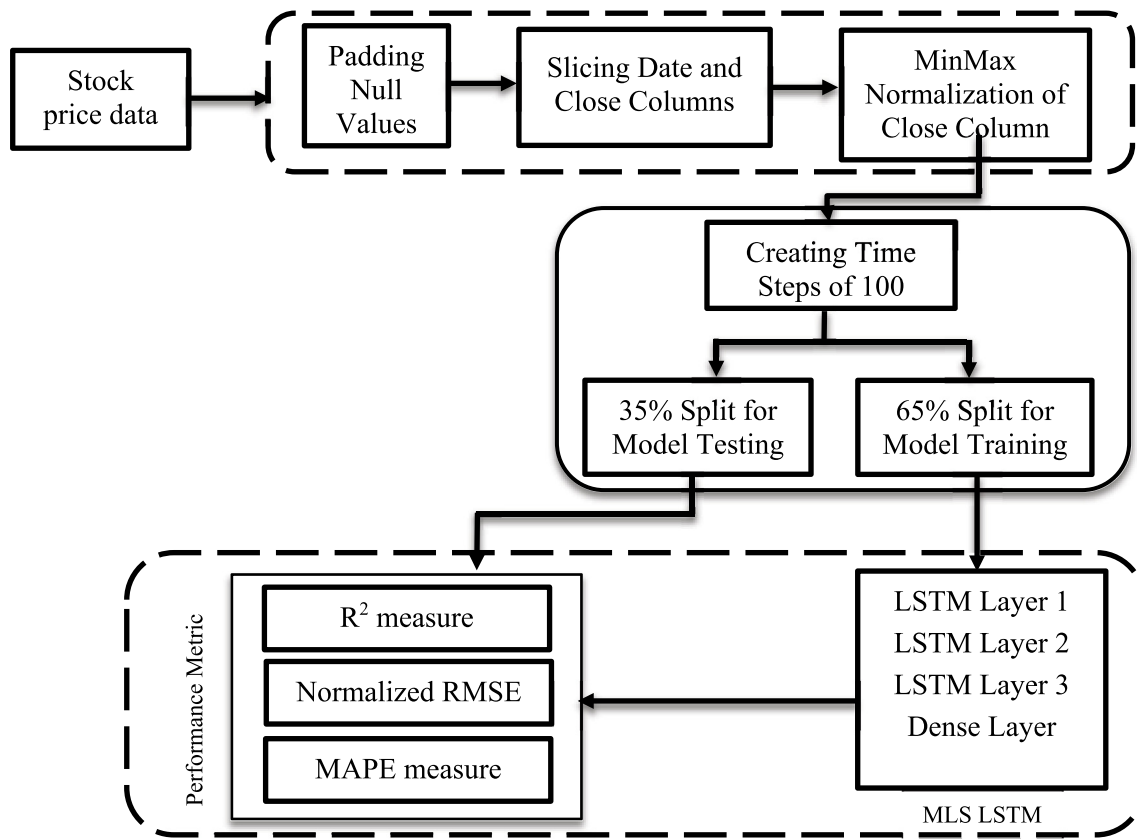


Fig. 5. Proposed architecture of multi-layer sequential long short-term memory.

Table 1

Stock closing price with date.

Date	Close
2017-11-24	55460
2018-03-19	50740
2019-01-17	41950
2019-07-04	46000
2020-12-04	71500

4.3. Visualizing data

The next step is the plotting of the closing price of the stock over the time period of the dataset. The date is plotted along the x-axis and the closing price of the stock along the y-axis. Fig. 7 shows the line plot which gives a fair idea about the progression of the value of the Samsung stock over time starting from November 2016 to November 2021. Next, the maximum and minimum closing prices of the stock along with the date of the valuation are extracted from the data and five random values are shown in Table 1.

It is observed that the lowest price of the stock was 32980 on 23-11-2016, which is the starting point of the chosen dataset. The maximum valuation of the stock was observed on 01-11-2021 with a closing price of 91,000, which is a little under three times the starting price (also the lowest valuation) of the Samsung stock.

4.4. Data normalization

After the visualization of the dataset, feature scaling is performed on it. Feature scaling helps in normalizing all the data in the dataset using Eq. (6) which may fall between different ranges

and converts them into a new range of any given choice, which is usually 0 to 1. If the data is not normalized, it may lead to large gradient error values which will result in very large, and thus unstable, weight values for the LSTM model. To normalize the data, MinMaxScaler is used, which is available in the sklearn preprocessing library. It helps in scaling all the columns of the data between the value of 0 and 1. MinMax scaler is chosen for this purpose because it preserves the shape of the dataset, without any distortion. The transformation is given by the following equation:

$$X_{scaled} = \frac{(X - X_{min})}{(X_{max} - X_{min})} \quad (6)$$

In the next step of the process, the data is split into training and testing sets. The data is split into the above-mentioned parts in the ratio of 65:35. It implies that 65% of the data is used for training the LSTM model and 35% is used for testing.

4.5. Reshaping data

To make the data suitable for the LSTM model, it is converted into time steps itself. The step value is taken to be 100. After that, the testing and training data are reshaped to fit the LSTM model that is going to be built. The reshape function takes three parameters, namely, the sample size, the time step and the number of features. For the proposed LSTM algorithm, the sample size is the number of rows in the training set, which is 329. The time step, as has already been mentioned, is 100. Finally, the feature size of the data is going to be 1, because only one feature, the closing price of the stock, is being predicted.

Algorithm 2: Building MLS (Multi-Layered Sequential) LSTM model**Step 1: Data Collection**

Read stock_price

Step 2: Data Preparation

For each value in data:

If value==null:

Replace_null(method='pad')

Separate 'Date' and 'Close' columns

MinMaxScaler(data)

Split data into Training and Testing set

Reshape_data(timestep=100)

Step 3: Build Prediction Model

Add layers to LSTM

Layer 1: return_sequence = True, input_size = (100,1)

Layer 2: return_sequence = True

Layer 3: return_sequence = False

Layer 4: Dense layer (1)

Loss function = mean-squared error

Optimizer = adam

Step 4: Train Model

Fit Training data on model

epochs = 100

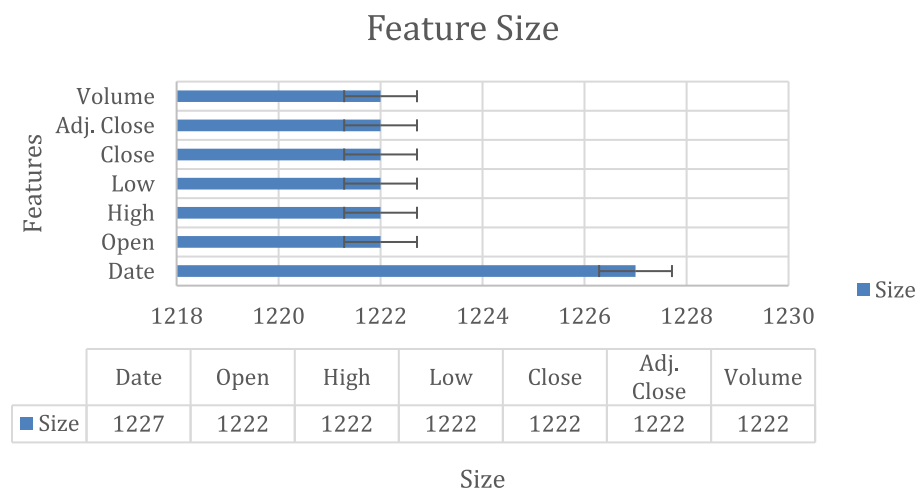
batch_size = 64

Step 5: Check Accuracy

R2_score()

root_mean_squared_error()

mean_absolute_percentage_error()

**Fig. 6.** Feature size of the data set.**4.6. Building MLS LSTM model**

The next step is building the MLS LSTM model for the prediction of stock prices. The type of model is set to sequential, which means that the data is fed to all the layers of the LSTM model in a sequential way. The MLS LSTM consists of 4 layers, which was found to be the optimal number of layers for accurate predictions while avoiding overfitting. The first layer of the model is the LSTM model that is imported from the Keras library. The input shape of the layer is defined as (100,1) because of the chosen time-step of 100. It signifies that each input is of size 100 and there is 1 feature

column that is being passed as the input. The number of LSTM cells is set as 50. The second layer of the model is also an LSTM layer of shape (50) and in the same way, the third layer is also an LSTM layer with shape (50). The final layer is the dense layer of shape (1). This indicates that all the output layers of the previous layer are densely connected (fully connected) to the output layer of the model. This results in a single output from all the cells of the LSTM cells. The optimizer used is adam, the working of which is illustrated in Algorithm 3.

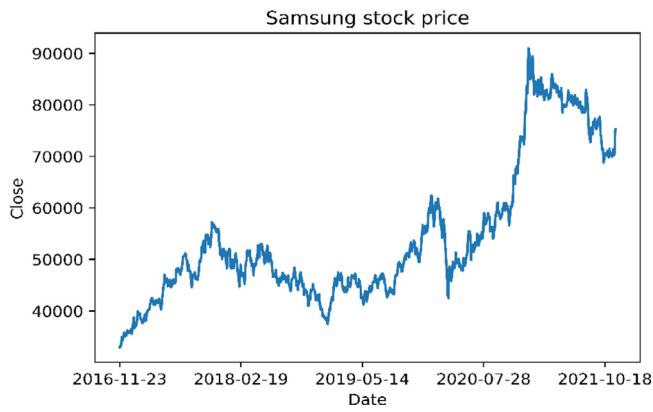


Fig. 7. Samsung stock price over a time period.

4.7. Model compilation and fitting

After the model is built, it is compiled with a batch size of 64 and an epoch size of 100. It signifies that the data is grouped together into batches of 64 and fed to the LSTM model. 100 epochs mean that the data is fed to the model 100 times. The validation split is 0.1, which means 10% of the training data is reserved for the validation of the model. After the training, predictions are made using both, training and testing data. The predicted data needs to be normalized back to the normal scale, which is done using the `inverse_transform()` function. After the data is inverse-scaled, it can be compared with the original data to obtain the accuracy measures. The accuracy measures used to gauge the performance of the model are the R^2 score, RMSE score and MAPE.

5. Experimental setup and results

The experiments were carried out on an i7-8750H Intel Core CPU with a 2.2 GHz machine with 8 GB ram and were implemented in Python 3.9.8 using Jupyter Notebook server 6.4.5. The dataset used in the experiment is that of Samsung's stock prices recorded between 23 November 2016 and 23 November 2021, ranging over five years. The dataset was obtained from the website of Yahoo Finances. Out of the seven columns in the dataset, only the Date column and the Closing price column are selected for further steps. The missing values are handled by replacement with the values from the previous row. For efficient training of the model, the data is normalized using the MinMax scaling technique. The training set is then split into 7 sets, while the testing set is divided into 10 sets. Both the training and testing sets are converted into time steps of 100. The input data is then reshaped as [samples, time steps, features]. Here, samples represent the number of rows in the data, the time step is 100 and features represent the number of features in every time step, which is 1.

For the prediction, a sequential LSTM model comprising three stacked LSTM layers is used. This is done with the help of Keras deep learning API that runs on top of TensorFlow. Finally, a dense layer is added to make the output a single value. During the compilation of the model, the loss function that is used is mean squared error and the optimizer is adam. The training of the model is performed for 100 epochs with 64 being the batch size and 0.1 being the validation split. The predictions are made on training and testing data using the compiled model. After the predictions are made, the original values and the predicted values are inverse-transformed back to their original scale for evaluation of their performance.

Table 2
Performance metrics.

Metric	Training data	Testing data
R^2	0.959	0.981
Adjusted R^2	0.958	0.979
Norm. RMSE	0.019	0.028
MAPE	1.79	2.18

5.1. Performance evaluation

The training as well as the testing data are used for the forecasting of the stock prices and are compared with the original values. The first metric used to measure the accuracy of the proposed algorithm is the R^2 score, which is defined as the proportion of the variance in the dependent variable that is predictable from the independent variables. R^2 is given by Eq. (7):

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y}_i)^2} \quad (7)$$

The R^2 score of the proposed model on training data was calculated to be 0.959 and that on testing data was calculated to be 0.981, which means that it performed with an accuracy of 95.9% on training data and with an accuracy of 98.1% on testing data. Adjusted R^2 takes into consideration the number of terms used in a model. It penalizes the addition of independent variables for prediction, which means that the Adjusted R^2 score decreases if useless variables are added. Adjusted R^2 can never be more than R^2 , it is always less than or equal to R^2 . The adjusted R^2 is calculated by the given Eq. (8):

$$R_{adj}^2 = 1 - \left[\frac{(1 - R^2)(n - 1)}{n - k - 1} \right] \quad (8)$$

The adjusted R^2 of the proposed MLS LSTM model is 0.958 on the training data and 0.979 on the testing data. As a second metric for measuring the performance of the model, the normalized root mean square error score (RMSE) is used, which is a value between 0 and 1 with 0 being the most favourable value. The formula to calculate the RMSE score is shown in Eq. (9). The Normalized RMSE score of the model on train data is 0.019 and on test data is 0.028.

$$Norm.RMSE = \frac{\sqrt{\frac{\sum (\hat{y}_i - y_i)^2}{n}}}{y_{max} - y_{min}} \quad (9)$$

The Mean Absolute Percentage Error (MAPE) gives information about how accurate a prediction model is. It is measured in percentage, and its value can be found by calculating the average modulus of the actual value of observation minus forecasted observation divided by the actual value of observation as shown in Eq. (10).

$$MAPE = \frac{1}{n} \sum_{t=1}^n \frac{|A_t - F_t|}{A_t} \quad (10)$$

The performance metrics, namely R^2 , Adjusted R^2 , Normalized RMSE and MAPE for the MLS LSTM are computed for training and testing data respectively, which are illustrated in Table 2. The values are then plotted in Fig. 8.

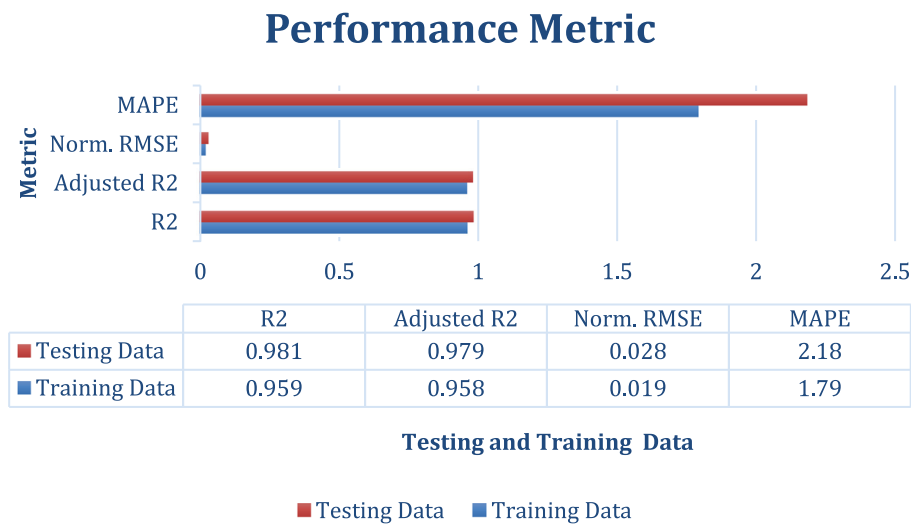
The actual stock price on a given date and the predicted stock price are tabulated in Table 3 and visualized in Fig. 9 to highlight the accuracy of forecasting by MLS LSTM. It can be observed that there is a very high overlap between the actual stock price and the stock prices predicted by the model, which means that the predictions are highly accurate.

Algorithm 3: Adam optimizer**Constrain:** α : size of step**Constrain:** $\eta_1, \eta_2 \in [0, 1)$: fractional decline proportions for the estimates moment**Constrain:** $x(\Theta)$: Stochastic function objective (parameter Θ)**Parameter Initialization :** Θ_0 :

$r_0 = 0$ (First moment)
 $q_0 = 0$ (Second moment)
 $j = 0$ (Time step)

while Θ_i not converged **do**

$j = j+1$
 $u_j = \nabla_{\Theta} x_t(\Theta_{j-1})$ (Acquire gradients: at time step j for stochastic objective)
 $r_j = \eta_1 * r_{j-1} + (1 - \eta_1) * u_j$ (Updating the biased - 1st moment approximation)
 $q_j = \eta_2 * q_{j-1} + (1 - \eta_2) * u_j^2$ (Updating the biased - 2nd raw moment approximation)
 $\tilde{r}_j = r_j / (1 - \eta_1^j)$ (Calculate corrected bias - 1st moment approximation)
 $\tilde{q}_j = q_j / (1 - \eta_2^j)$ (Calculate corrected bias - 2nd raw moment approximation)
 $\Theta_j = \Theta_{j-1} - \alpha * \tilde{r}_j / (\sqrt{\tilde{q}_j} + \epsilon)$ (parameters update)

end while**return** Θ_j (parameter Outcome)**Fig. 8.** Performance metrics.**Table 3**

Comparison of actual price with the predicted price.

Actual value	Predicted value
48251	47580
48093	47420
50664	49900
44504	44000
47034	45100

Table 4Training data R^2 comparison of different prediction techniques at multiple epochs.

Epoch count	Linear regression	SVM	MLS LSTM
5	0.39	0.39	0.42
30	0.60	0.75	0.79
45	0.64	0.80	0.83
75	0.69	0.88	0.90
100	0.73	0.93	0.95

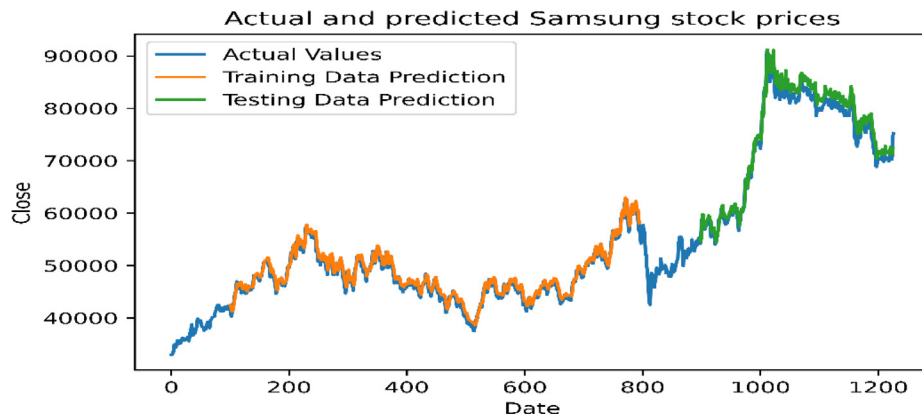


Fig. 9. Actual and predicted values of Samsung stock price.

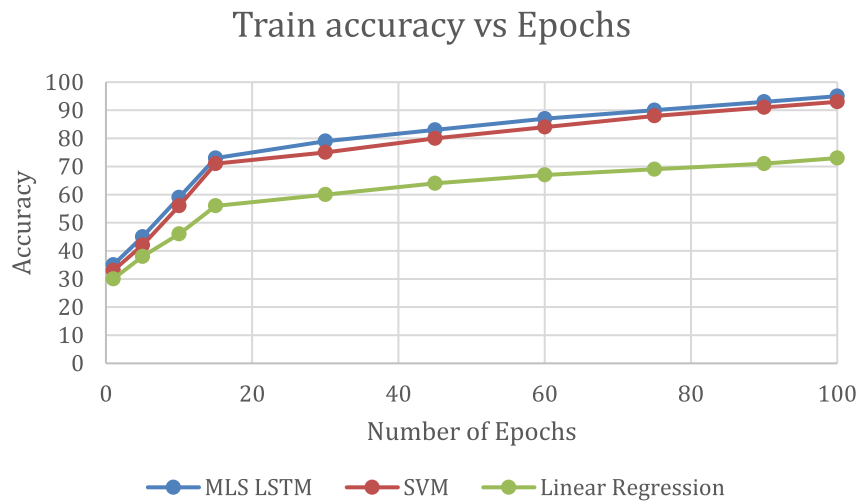


Fig. 10. Training data R^2 comparison.

Table 5
Testing data R^2 comparison of different prediction techniques at multiple epochs.

Epoch count	Linear regression	SVM	MLS LSTM
5	0.45	0.42	0.38
30	0.58	0.77	0.77
45	0.61	0.81	0.84
75	0.68	0.87	0.93
100	0.72	0.92	0.98

5.2. Performance comparison

In [2], the authors have used linear regression and support vector machines for the prediction of stock prices. The comparison of the R^2 score of the linear regression and the support vector machine model is performed with the proposed MLS LSTM model in Tables 4 and 5. The R^2 of the linear regression model trained by the authors has an R^2 score of 0.73.

The SVM model is better performing, with an R^2 score of 0.93. But the proposed MLS LSTM approach outperforms both, the regression model and the SVM model, with an R^2 score of 0.98. The epoch-wise values are plotted in Figs. 10 and 11. Based on this, it can be shown that MLS LSTM can produce more precise stock price predictions and predict values with higher accuracy when compared to simple machine learning approaches such as SVM and linear regression.

In [7], the authors made use of four different deep neural networks, namely LSTM, MLP, RNN and CNN to compare the performance of each of the techniques on three different stocks

Table 6
MAPE comparison of different prediction techniques at multiple epochs.

Epoch count	RNN	LSTM	CNN	MLP	MLS LSTM
15	8.49	6.31	5.32	5.49	3.11
30	8.32	6.12	5.19	5.35	2.97
45	8.17	5.99	5.04	5.21	2.79
60	8.01	5.83	4.93	5.08	2.52
75	7.88	5.69	4.78	4.93	2.39
90	7.73	5.55	4.67	4.82	2.27
100	7.62	5.47	4.55	4.72	2.18

(Maruti, HCL and Axis Bank) for the duration of 400 days. To determine and compare the proposed MLS LSTM performance with theirs, the average MAPE value of the three stocks obtained by the authors is compared.

It can be observed that RNN has the largest MAPE value out of all the deep learning algorithms, which can be attributed to its inability to account for long-term dependencies in data. It is also observed that the MAPE value of the proposed MLS LSTM

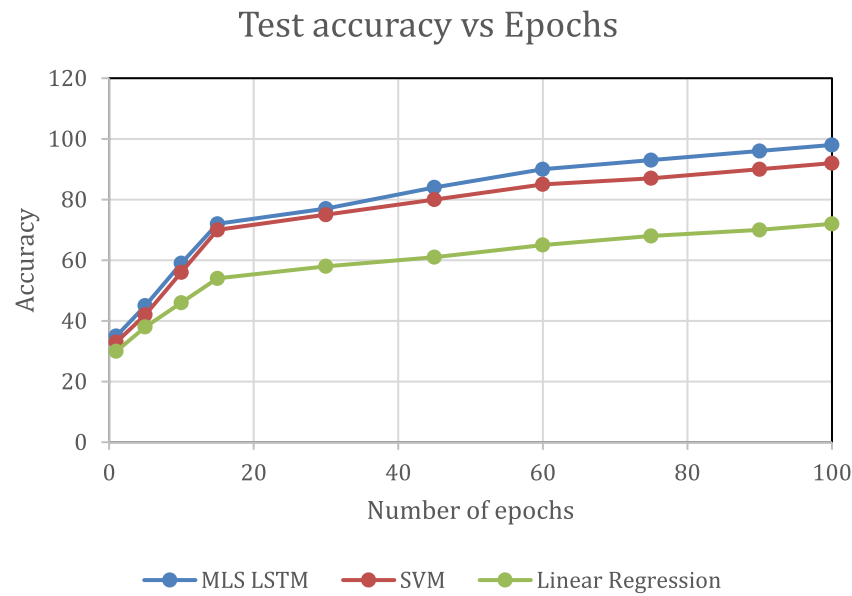


Fig. 11. Testing data R^2 comparison.

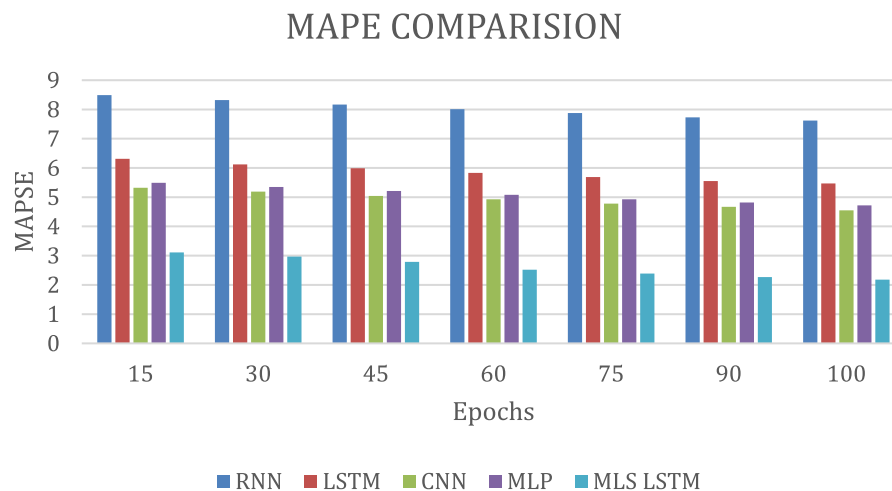


Fig. 12. MAPE comparison.

model is lower than any of the other deep learning architectures, with a mean error percentage of only 2.18% on the testing data. It can be noted that the MAPE of the proposed MLS LSTM is lower than a single-layer LSTM, thus showing that a multi-layered LSTM performs better than a single-layered LSTM. This can be attributed to the fact that each additional layer of the LSTM fine-tunes the weights to reach the optimally accurate solution. The MAPE values are illustrated in Table 6 and visualized in Fig. 12.

6. Conclusion and future work

The paper proposes a new approach for stock price prediction based on a Multi-Layer Sequential Long Short Term Memory (MLS LSTM) model that makes use of the Adam optimizer. As a result of the MLS LSTM algorithm, normalized time series data is divided into time steps to determine the relationship between past and future values for accurate prediction. In the testing data set, MLS LSTM achieved a prediction accuracy of 95.9% and 98.1% on the testing data set, outperforming every other deep learning and machine learning algorithm currently available. The average error percentage with which the proposed model is able to predict the values is 2.18%, which is considerably low, thus giving very

accurate predictions. MLS LSTM is a better approach also because more layers in the model reduce the generality of the results.

In an effort to build a more efficient and reliable prediction system in the future, fundamental analysis and sentiment analysis will be included in the prediction process. By doing this, we will be able to gauge the general sentiment of the public regarding the company, which will allow us to tweak the prediction technique based on the sentiment and make even more accurate predictions in the future. Changing the way the missing data is handled, that is, replacing null values with running averages or median of a set range of data can also help increase training accuracy. An effort to reduce the training time as well as resource requirements of LSTM must be made. The problem of overfitting must also be taken care of, as LSTM can be sensitive to it.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] R. Seethalakshmi, Analysis of stock market predictor variables using linear regression, *Int. J. Pure Appl. Math.* 119 (15) (2018) 369–378.
- [2] F.S. Gharehchopogh, T.H. Bonab, S.R. Khaze, A linear regression approach to prediction of stock market trading volume: a case study, *Int. J. Manag. Value Supply Chains* 4 (3) (2013) 25.
- [3] V. Gururaj, V.R. Shriya, K. Ashwini, Stock market prediction using linear regression and support vector machines, *Int. J. Appl. Eng. Res.* 14 (8) (2019) 1931–1934.
- [4] N.I. Sapankevych, R. Sankar, Time series prediction using support vector machines: a survey, *IEEE Comput. Intell. Mag.* 4 (2) (2009) 24–38.
- [5] P. Mondal, L. Shit, S. Goswami, Study of effectiveness of time series modeling (ARIMA) in forecasting stock prices, *Int. J. Comput. Sci. Eng. Appl.* 4 (2) (2014) 13.
- [6] A.S. Saud, S. Shakya, Analysis of look back period for stock price prediction with RNN variants: A case study on banking sector of NEPSE, *Procedia Comput. Sci.* 167 (2020) 788–798.
- [7] M.O. Rahman, M.S. Hossain, T.S. Junaid, M.S.A. Forhad, M.K. Hossen, Predicting prices of stock market using gated recurrent units (GRUs) neural networks, *Int. J. Comput. Sci. Netw. Secur.* 19 (1) (2019) 213–222.
- [8] M. Roondiwala, H. Patel, S. Varma, Predicting stock prices using LSTM, *Int. J. Sci. Res. (IJSR)* 6 (4) (2017) 1754–1756.
- [9] K. Pawar, R.S. Jalem, V. Tiwari, Stock market price prediction using LSTM RNN, in: *Emerging Trends in Expert Applications and Security*, Springer, Singapore, 2019, pp. 493–503.
- [10] E. Hoseinzade, S. Haratizadeh, CNNpred: CNN-based stock market prediction using a diverse set of variables, *Expert Syst. Appl.* 129 (2019) 273–285.
- [11] K. Pothuganti, Long short-term memory (LSTM) algorithm based prediction of stock market exchange, *Int. J. Res. Publ. Rev.* 2 (1) (2021) 90–93.
- [12] A. Moghar, M. Hamiche, Stock market prediction using LSTM recurrent neural network, *Procedia Comput. Sci.* 170 (2020) 1168–1173.
- [13] M. Hiransha, E.A. Gopalakrishnan, V.K. Menon, K.P. Soman, NSE stock market prediction using deep-learning models, *Procedia Comput. Sci.* 132 (2018) 1351–1362.
- [14] A.V. Devadoss, T.A.A. Ligor, Forecasting of stock prices using multi layer perceptron, *Int. J. Comput. Algorithm* 2 (1) (2013) 440–449.
- [15] K. Khare, O. Darekar, P. Gupta, V.Z. Attar, Short term stock price prediction using deep learning, in: *2017 2nd IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology, RTEICT, IEEE, 2017*, pp. 482–486.
- [16] H. Jia, Investigation into the effectiveness of long short term memory networks for stock price prediction, 2016, arXiv preprint arXiv:1603.07893.
- [17] K. Chen, Y. Zhou, F. Dai, A LSTM-based method for stock returns prediction: A case study of China stock market, in: *2015 IEEE International Conference on Big Data, big data, IEEE, 2015*, pp. 2823–2824.
- [18] F. Jiang, G. Tang, G. Zhou, Firm characteristics and Chinese stocks, *J. Manage. Sci. Eng.* 3 (4) (2018) 259–283.
- [19] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, *J. Mach. Learn. Res.* 13 (2) (2012).
- [20] A. Yadav, C.K. Jha, A. Sharan, Optimizing LSTM for time series prediction in Indian stock market, *Procedia Comput. Sci.* 167 (2020) 2091–2100.